KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Artificial Intelligence and Data Science

Academic Year 2026 – 2027 (Odd Semester)

U21ADP05 - Exploratory Data Analysis and Visualization

Assignment II - Case Study Project Report

Exploratory Data Analysis and Visualization of the Breast Cancer Wisconsin (Diagnostic) Dataset

with a Deep Learning (MLP) Classifier

Item	Details
Student Name	Poorvisha S
Roll No.	24AD078
Department	Artificial Intelligence and Data Science
Year / Semester	III Year / V Semester (Batch 2024-2028)
GitHub Repository	https://github.com/Poorvisha-05/EDA_Assignment

Abstract

Early and accurate classification of breast masses as malignant or benign supports timely clinical decisions. This case study applies exploratory data analysis (EDA) and visualization to the Breast Cancer Wisconsin (Diagnostic) dataset, containing 569 patients and 30 numerical features computed from digitized fine-needle-aspirate images. The data were checked for missing values, duplicates and outliers, standardized, and split into train, validation and test sets. Seventeen visualizations covering all seven visualization stages revealed strong class separation and heavy multicollinearity among size-related features. A multilayer perceptron with hyperparameter tuning achieved 96.5% test accuracy and an AUC of 0.990. Size and concavity features proved most informative, and a logistic regression baseline reached 98.8% accuracy.

Keywords: exploratory data analysis, data visualization, breast cancer, multilayer perceptron, ROC-AUC, feature importance

1. Introduction and Objective

1.1 Background and motivation

Breast cancer is one of the most common cancers worldwide, and the outcome for a patient depends strongly on how early and how reliably a tumour is identified as malignant. Fine-needle aspiration (FNA) is a minimally invasive procedure in which cells are extracted from a breast mass and examined under a microscope. Digitizing the slide image and measuring the shape of each cell nucleus gives quantitative features that can be analysed statistically and used to train predictive models [1].

Real-world data of this type is rarely ready for modelling: features are on different scales, strongly correlated, skewed, and contain extreme values. Exploratory data analysis (EDA), a discipline introduced by Tukey [8], and visualization are therefore the first and most important step, because they reveal what the data can and cannot support before any model is trained. A model trained without this understanding may look accurate while hiding data leakage, class imbalance or redundant inputs.

1.2 Problem statement

Given thirty numerical measurements describing the cell nuclei of a breast mass, the task is to (i) understand the structure, quality and statistical behaviour of the data through EDA and visualization, and (ii) build and evaluate a deep learning classifier that predicts whether the mass is malignant or benign. The case study is deliberately framed around the practical application of decision support for cytology screening, where the cost of a missed malignant case is far greater than the cost of a false alarm.

1.3 Objectives

- Select a real-world dataset with at least 15 features and describe its source, size and fields.
- Perform EDA: handle missing values, duplicates and outliers; transform, scale and split the data.
- Create at least five meaningful visualizations covering the seven stages of visualization, each explained.
- Analyse univariate distributions (discrete and continuous), and bivariate / multivariate relationships.
- Build, tune and evaluate a deep learning model (MLP) and interpret loss, accuracy, confusion matrix, ROC and AUC.
- Draw practical conclusions and suggest future improvements.

1.4 Scope and contributions

The study is limited to the tabular WDBC features (not the original images) and to a single train/validation/test split. Its contributions are: (1) a reproducible, well-commented Python pipeline published on GitHub; (2) seventeen interpreted visualizations and a full per-feature statistical profile; (3) a tuned MLP compared against logistic regression and random forest

baselines; and (4) an error analysis that goes beyond headline accuracy to threshold behaviour and individual misclassified patients.

1.5 Organisation of the report

Section 2 reviews related work and Section 3 summarises the statistical and deep-learning theory used. Section 4 describes the methodology and tools. Sections 5 and 6 present the dataset and its EDA and preprocessing. Section 7 contains the visualizations, Sections 8 and 9 the model and its results, Section 10 a discussion, Section 11 limitations and ethics, and Section 12 the conclusions and future scope.

2. Literature Review

2.1 Exploratory data analysis and visualization

Tukey [8] argued that analysts should examine data with simple numerical and graphical summaries before committing to a formal model, so that unexpected structure such as outliers, skewness or clusters is discovered rather than assumed away. Fry [6] organised the process of turning raw data into a visual into seven stages - acquire, parse, filter, mine, represent, refine and interact - which is the framework used to structure the visualizations in this report. Modern Python libraries such as Matplotlib [10] and Seaborn [11] make these steps reproducible in code.

2.2 The Wisconsin breast-cancer data and related work

The WDBC data originate from work at the University of Wisconsin. Street, Wolberg and Mangasarian [1] described an interactive image-analysis system that locates cell nuclei in digitized FNA slides and computes quantitative shape and texture features from each nucleus. Earlier and later work by the same group applied linear-programming-based separation methods to cytology features for diagnosis and prognosis [9], [12]. The dataset was subsequently deposited in the UCI Machine Learning Repository [2] and, because it is small, clean and well documented, has become a standard benchmark for classification algorithms and teaching material.

2.3 Models used in this study

Multilayer perceptrons are the classic feed-forward neural network [7]. Their generalisation is improved by regularisation methods such as dropout [5] and by early stopping, and they are commonly trained with the Adam optimiser [4]. Random forests and regularised linear models are standard baselines for tabular data [13]; they are used here to judge whether the added complexity of a neural network is justified.

2.4 Summary of the reviewed literature

Table 1: Summary of the main references

Ref.	Type	Contribution	Use in this project
[1]	Conference paper	Nuclear feature extraction for breast tumour diagnosis	Origin and meaning of the 30 features
[2]	Data repository	UCI repository entry for WDBC	Dataset source and documentation
[3]	Journal paper	Scikit-learn library	Dataset loading, scaling, metrics, baselines
[4]	Preprint	Adam optimiser	Training algorithm of the MLP
[5]	Journal paper	Dropout regularisation	Overfitting control
[6]	Book	Seven stages of visualization	Structure of Section 7
[7]	Book	Deep Learning (Goodfellow et al.)	MLP, loss and optimisation theory
[8]	Book	Exploratory Data Analysis (Tukey)	Philosophy of EDA
[9], [12]	Journal papers	Linear-programming diagnosis from cytology data	Context of earlier work on the same data
[13]	Book	Elements of Statistical Learning	Random forest, regularisation, evaluation

2.5 Research gap and positioning

Many published uses of this dataset report a single accuracy figure. This case study instead emphasises the full analytical workflow: data quality checks, distribution analysis, multicollinearity, feature importance, threshold analysis and error analysis, so that a reader can see not only how well a model performs but why, and where it fails.

3. Theoretical Background

This section summarises the statistical and machine-learning concepts used in the analysis so that the results in later sections can be interpreted correctly.

3.1 Descriptive statistics

For a feature x with n observations, the mean and sample standard deviation describe location and spread. The skewness measures asymmetry and the excess kurtosis measures tail weight; a normal distribution has skewness 0 and excess kurtosis 0. As a rule of thumb, |skewness| > 1 indicates strong skew.

$$\text{mean} = \frac{1}{n}\sum x_i \qquad s = \sqrt{\sum (x_i - \text{mean})^2 / (n-1)} \qquad \text{skewness} = \frac{1}{n}\sum \left(\frac{x_i - \text{mean}}{s}\right)^3$$

The interquartile range IQR = Q3 − Q1 is robust to extreme values. Tukey's rule marks an observation as an outlier if it lies below Q1 − 1.5·IQR or above Q3 + 1.5·IQR; values beyond 3·IQR are considered extreme and were used as clipping fences in this project.

3.2 Probability distributions

The diagnosis of one patient is a Bernoulli random variable with success probability p (here, benign). The number of benign cases among n patients follows a Binomial(n, p) distribution with mean np and variance np(1−p). Continuous measurements are often modelled with the normal distribution; positive, right-skewed measurements such as area are frequently better described by the log-normal distribution, whose logarithm is normal.

$$\text{Bernoulli: } P(X=1) = p \qquad \text{Binomial: } P(X=k) = \binom{n}{k} p^k (1-p)^{n-k} \qquad \text{Normal: } f(x) = \frac{\exp(-(x-\mu)^2 / 2\sigma^2)}{\sigma\sqrt{2\pi}}$$

Normality can be assessed visually (histogram with fitted curve, Q-Q plot) and formally with the Shapiro-Wilk test, whose null hypothesis is that the sample comes from a normal distribution. Two fitted distributions can be compared with the Kolmogorov-Smirnov (KS) statistic, the maximum vertical distance between the empirical and the fitted cumulative distribution function; a smaller value indicates a better fit.

3.3 Relationships between variables

The Pearson correlation coefficient r measures the strength of a linear relationship between two variables (−1 to +1). Strongly correlated predictors (multicollinearity) carry redundant information. Mutual information (MI) measures how much knowing a feature reduces uncertainty about the class and, unlike correlation, also captures non-linear dependence. To compare group means, Welch's t-test does not assume equal variances, and Cohen's d expresses the difference in pooled standard-deviation units (|d| ≈ 0.2 small, 0.5 medium, 0.8 large).

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \qquad d = \frac{\text{mean}_1 - \text{mean}_2}{s_{\text{pooled}}}$$

3.4 Principal component analysis

PCA rotates standardized data to new, uncorrelated axes (principal components) ordered by the amount of variance they explain. It is used here to reveal how many independent directions are really present in the 30 correlated features and to draw a two-dimensional map of the patients.

3.5 Multilayer perceptron

An MLP is a stack of fully connected layers. Each neuron computes a weighted sum of its inputs plus a bias and passes it through a non-linear activation. The hidden layers use the rectified linear unit (ReLU), which is cheap to compute and avoids vanishing gradients; the single output neuron uses the sigmoid function, which maps any real number to a probability between 0 and 1.

$$ReLU(z) = max(0, z) \qquad \sigma(z) = 1 / (1 + e^{-z}) \qquad a^{(l)} = f(W^{(l)} a^{(l-1)} + b^{(l)})$$

The network is trained by minimising the binary cross-entropy loss with back-propagation of gradients. Cross-entropy strongly penalises confident wrong predictions.

$$L = -(1/n) \Sigma [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

The Adam optimiser [4] keeps running averages of the gradient and its square for every weight and scales the learning rate accordingly, which typically gives fast and stable convergence. Dropout [5] randomly sets a fraction of hidden activations to zero during training, preventing neurons from co-adapting and so reducing overfitting. Early stopping monitors the validation loss and stops training when it no longer improves, restoring the best weights.

3.6 Evaluation metrics

For a binary classifier, the confusion matrix counts true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN). In this project the 'positive' class is Benign (label 1), so an FP is a malignant case wrongly called benign - the most dangerous error.

$$Accuracy = (TP + TN)/n \qquad Precision = TP/(TP + FP) \qquad Recall = TP/(TP + FN) \qquad F1 = 2PR/(P + R) \qquad Specificity = TN/(TN + FP)$$

The ROC curve plots the true-positive rate against the false-positive rate for every decision threshold; the area under it (AUC) equals the probability that a randomly chosen positive case receives a higher score than a randomly chosen negative case. The precision-recall curve is more informative when classes are imbalanced.

4. Methodology and Tools

The work follows the pipeline shown in the figure below. Every step is implemented in Python scripts stored in the GitHub repository, so the whole study can be reproduced with three commands.

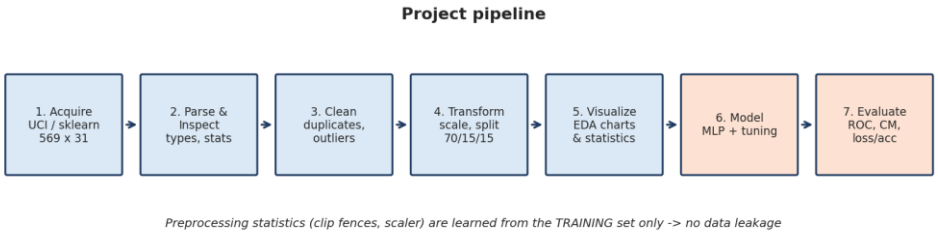


Figure 1: Overall methodology of the project

4.1 Tools and environment

Table 2: Tools used

Component	Tool / library	Purpose
Language	Python 3	All analysis and modelling code
Data handling	pandas, NumPy, SciPy	Tables, numerical operations, statistical tests
Preprocessing / baselines	scikit-learn [3]	Dataset loading, scaling, split, PCA, logistic regression, random forest, metrics
Visualization	Matplotlib [10], Seaborn [11], Plotly (optional)	Static and interactive charts
Deep learning	TensorFlow / Keras [14]	MLP model (with a NumPy fallback implementation)
Documentation	python-docx, GitHub	Report generation and version control

4.2 Experimental design

- **Splitting:** The dataset is split once, stratified by diagnosis, into 70% training, 15% validation and 15% test data with a fixed random seed (42).
- **Leakage control:** Outlier fences and the standardization parameters are computed on the training set only and then applied to validation and test data, which prevents information leakage.
- **Model selection:** Hyperparameters are chosen using the validation set only; the test set is used exactly once for the final evaluation.
- **Baselines:** The MLP is compared with logistic regression (linear baseline) and random forest (non-linear baseline).
- **Evaluation:** Accuracy, precision, recall, F1, ROC-AUC, average precision, confusion matrix and threshold behaviour are reported.

5. Dataset Description

The Breast Cancer Wisconsin (Diagnostic) dataset (WDBC) was created by Street, Wolberg and Mangasarian at the University of Wisconsin and is hosted in the UCI Machine Learning Repository [1], [2]. It is also bundled with scikit-learn [3], which was used to load it. Features describe characteristics of cell nuclei in a digitized FNA image of a breast mass.

Table 3: Dataset overview

Property	Value
Source	UCI Machine Learning Repository / scikit-learn (load_breast_cancer)
Records (patients)	569
Input features	30
Target	diagnosis: 0 = Malignant, 1 = Benign
Class counts	Malignant = 212 (37.3%), Benign = 357 (62.7%)
Feature type	All continuous (real-valued)
Missing values	0
Duplicate rows	0

5.1 Feature dictionary

Ten base measurements are computed for every nucleus. For each, three versions are stored - the mean over all nuclei in the image, the standard error (SE) and the 'worst' (mean of the three largest values) - giving 10 × 3 = 30 features.

Table 4: Description of the 30 features

Base measurement	Meaning	Stored as
Radius	Mean distance from the centre to points on the nucleus boundary	mean radius, radius error, worst radius
Texture	Standard deviation of gray-scale values	mean texture, texture error, worst texture
Perimeter	Length of the nucleus boundary	mean perimeter, perimeter error, worst perimeter
Area	Area enclosed by the nucleus boundary	mean area, area error, worst area
Smoothness	Local variation in radius lengths	mean smoothness, smoothness error, worst smoothness
Compactness	perimeter² / area − 1.0	mean compactness, compactness error, worst compactness
Concavity	Severity of concave portions of the contour	mean concavity, concavity error, worst concavity
Concave points	Number of concave portions of the contour	mean concave points, concave points error, worst concave points
Symmetry	Symmetry of the nucleus shape	mean symmetry, symmetry error, worst symmetry
Fractal dimension	'Coastline approximation' − 1; roughness of the boundary	mean fractal dimension, fractal dimension error, worst fractal dimension

5.2 Basic statistics

Table 5: Basic statistics of selected features (all 30 in results/summary_statistics.csv)

Feature	Mean	Std	Min	Median	Max
mean radius	14.127	3.524	6.981	13.370	28.110
mean texture	19.290	4.301	9.710	18.840	39.280
mean perimeter	91.969	24.299	43.790	86.240	188.500
mean area	654.889	351.914	143.500	551.100	2501.000
mean smoothness	0.096	0.014	0.053	0.096	0.163
mean compactness	0.104	0.053	0.019	0.093	0.345
mean concavity	0.089	0.080	0.000	0.062	0.427
mean concave points	0.049	0.039	0.000	0.034	0.201
worst area	880.583	569.357	185.200	686.500	4254.000
area error	40.337	45.491	6.802	24.530	542.200

The ranges differ by orders of magnitude: area is measured in hundreds to thousands of units whereas smoothness is about 0.1. This is the main reason why scaling is necessary before training a neural network.

5.3 Class-wise comparison of the mean features

Table 6: Mean value of each 'mean' feature by diagnosis

Feature	Malignant mean	Benign mean	Malignant / Benign
mean radius	17.4628	12.1465	1.44
mean texture	21.6049	17.9148	1.21
mean perimeter	115.3654	78.0754	1.48
mean area	978.3764	462.7902	2.11
mean smoothness	0.1029	0.0925	1.11
mean compactness	0.1452	0.0801	1.81
mean concavity	0.1608	0.0461	3.49
mean concave points	0.0880	0.0257	3.42
mean symmetry	0.1929	0.1742	1.11
mean fractal dimension	0.0627	0.0629	1.00

Malignant nuclei are on average 2.11 times larger in area and have 3.42 times more concave points than benign nuclei. These two ratios already show that shape irregularity and size are the strongest signs of malignancy in this data.

6. EDA and Preprocessing

6.1 Methods used

- **Loading and inspection:** Dataset loaded into a pandas DataFrame (569 x 31); data types, shape and summary statistics inspected.
- **Missing values and duplicates:** No missing values (0) and no duplicate rows (0) were found, so no imputation or removal was needed.
- **Outliers:** The 1.5 x IQR rule flagged 608 values; the most affected were area error (65), radius error (38), perimeter error (38), worst area (35), smoothness error (30). Because such values correspond to genuinely large tumours (clinically meaningful), they were not deleted. Instead, extreme values beyond 3 x IQR (fences computed on the training set only) were clipped - 82 values - to limit their influence on the neural network.
- **Encoding:** The target is already numerically encoded (0/1), and all 30 predictors are numerical, so no one-hot encoding was required.
- **Scaling:** Features were standardized to zero mean and unit variance with StandardScaler, fitted on the training set only to avoid data leakage.
- **Splitting:** A stratified 70/15/15 split gave 398 training, 85 validation and 86 test samples, preserving the class ratio in every subset.

6.2 Data quality assessment

The dataset is complete and free of duplicates. Outliers are, however, widespread: 29 of the 30 features contain at least one value outside the 1.5 x IQR fences, for a total of 608 flagged values (about 3.6% of all cells). The standard-error features are the most affected, probably because they measure variability within an image, which is naturally more erratic than a mean. Deleting these rows would remove many malignant patients, so clipping was preferred.

6.3 Complete per-feature statistical profile

The table below summarises, for every feature, its shape (skewness, kurtosis), number of IQR outliers, the Shapiro-Wilk and Welch t-test p-values, Cohen's d and mutual information with the diagnosis. Positive Cohen's d means the feature is larger in malignant tumours.

Table 7: Per-feature statistical profile

Feature	Skew	Kurt.	Outl.	Shapiro p	Welch p	Cohen d	MI
mean radius	0.94	0.85	14	<0.001	<0.001	2.21	0.362
mean texture	0.65	0.76	7	<0.001	<0.001	0.94	0.097
mean perimeter	0.99	0.97	13	<0.001	<0.001	2.29	0.402
mean area	1.65	3.65	25	<0.001	<0.001	2.08	0.360
mean smoothness	0.46	0.86	6	<0.001	<0.001	0.79	0.080
mean compactness	1.19	1.65	16	<0.001	<0.001	1.53	0.213
mean concavity	1.40	2.00	18	<0.001	<0.001	2.00	0.375
mean concave points	1.17	1.07	10	<0.001	<0.001	2.55	0.439
mean symmetry	0.73	1.29	15	<0.001	<0.001	0.72	0.066
mean fractal dimension	1.30	3.01	15	<0.001	0.767	-0.03	0.006
radius error	3.09	17.69	38	<0.001	<0.001	1.42	0.249
texture error	1.65	5.35	20	<0.001	0.835	-0.02	0.000
perimeter error	3.44	21.40	38	<0.001	<0.001	1.38	0.276
area error	5.45	49.21	65	<0.001	<0.001	1.35	0.341

smoothness error	2.31	10.47	30	<0.001	0.105	-0.14	0.016
compactness error	1.90	5.11	28	<0.001	<0.001	0.63	0.073
concavity error	5.11	48.86	22	<0.001	<0.001	0.54	0.117
concave points error	1.44	5.13	19	<0.001	<0.001	0.92	0.125
symmetry error	2.20	7.90	27	<0.001	0.887	-0.01	0.014
fractal dimension error	3.92	26.28	28	<0.001	0.042	0.16	0.039
worst radius	1.10	0.94	17	<0.001	<0.001	2.54	0.451
worst texture	0.50	0.22	5	<0.001	<0.001	1.06	0.120
worst perimeter	1.13	1.07	15	<0.001	<0.001	2.60	0.472
worst area	1.86	4.40	35	<0.001	<0.001	2.23	0.464
worst smoothness	0.42	0.52	7	<0.001	<0.001	0.96	0.096
worst compactness	1.47	3.04	16	<0.001	<0.001	1.51	0.225
worst concavity	1.15	1.62	12	<0.001	<0.001	1.81	0.315
worst concave points	0.49	-0.54	0	<0.001	<0.001	2.69	0.436
worst symmetry	1.43	4.44	23	<0.001	<0.001	0.95	0.095
worst fractal dimension	1.66	5.24	24	<0.001	<0.001	0.71	0.065

Findings: (i) 22 of 30 features have |skewness| > 1 and all 30 reject normality (Shapiro-Wilk $p < 0.05$); (ii) 20 features have a large effect size ($|d| > 0.8$), with the largest for worst concave points (2.69), worst perimeter (2.6), mean concave points (2.55); (iii) only four features show no significant class difference (mean fractal dimension, texture error, smoothness error, symmetry error) and these also have the lowest mutual information.

6.4 Distribution identification

Discrete: the diagnosis follows a Bernoulli distribution with p(benign) = 0.627. For a test set of 86 patients, the Binomial model predicts about 54 benign cases with a standard deviation of 4.48; the stratified test set contains exactly 54, so the split is representative. The probability of seeing at least 54 benign cases under this model is 0.54.

Continuous: normal and log-normal distributions were fitted to mean area, mean radius and mean concavity. The KS statistic for the normal fit was 0.156, 0.113 and 0.139, versus 0.063, 0.066 and 0.060 for the log-normal fit. The log-normal is therefore the better model for all three, in agreement with their right-skewed shape (the fitted curves are plotted in Section 7).

6.5 Dimensionality and redundancy

21 pairs of features have an absolute correlation above 0.9. For example, mean radius and worst radius correlate at $r = 0.97$. PCA on the standardized data shows that the first component alone explains 44.3% of the variance, the first two 63.2%, and 10 components are enough for 95%. The 30 features therefore contain only about ten independent directions of information.

6.6 Summary of EDA insights

- The classes are moderately imbalanced (about 63% benign, 37% malignant), so accuracy alone is not enough - recall, precision and AUC are also reported.
- Several features are strongly right-skewed (e.g., area error (5.45), concavity error (5.11), fractal dimension error (3.92)).
- Welch t-tests found a significant class difference ($p < 0.05$) for 26 of 30 features.
- Size features are almost perfectly correlated - mean radius | mean perimeter ($r = 0.998$); worst radius | worst perimeter ($r = 0.994$); mean radius | mean area ($r = 0.987$).
- Dimensionality is effectively about 10, not 30.

7. Data Visualization

Seventeen visualizations were produced, following the seven stages of data visualization described by Fry [6]. The table below maps each stage to the work done in the project. For every chart the report states what it shows, why it was created and what was learned from it.

Table 8: Mapping of the seven visualization stages

Stage	Activity in this project	Output
1. Acquire	Loaded WDBC dataset from scikit-learn / UCI and cached as CSV	data/breast_cancer_wisconsin.csv
2. Parse	Inspected structure, types, missing values, duplicates, summary statistics	summary_statistics.csv; Section 5
3. Filter	Removed duplicates (none), identified IQR outliers, kept clinically valid values	Section 6.2
4. Mine	Skewness, Shapiro-Wilk, t-tests, Cohen's d, correlation, mutual information, PCA	Sections 6.3 - 6.5
5. Represent	Chart type chosen by data type: bar, pie, histogram, KDE, box, violin, heatmap, scatter, pair, Q-Q, PCA	Sections 7.1 - 7.2
6. Refine	Titles, labels, colour-blind-friendly class colours, before/after scaling comparison	Scaling chart in Section 7.1
7. Interact	Interactive Plotly scatter (generated when plotly is installed)	results/interactive_scatter.html

7.1 Core visualizations

Class distribution

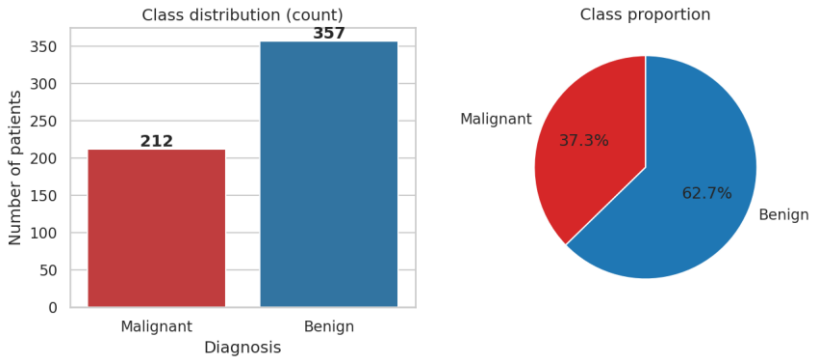


Figure 2: Class distribution

What it shows: A bar chart and pie chart of the number of malignant and benign patients (discrete target variable).

Why it was created: To check class balance before modelling, since imbalance affects the choice of evaluation metrics. The target follows a Bernoulli distribution for each patient and a Binomial distribution for counts.

Insights obtained: There are 357 benign (62.7%) and 212 malignant (37.3%) cases - moderately imbalanced but not severely, so stratified splitting is sufficient.

Histograms with fitted normal curve

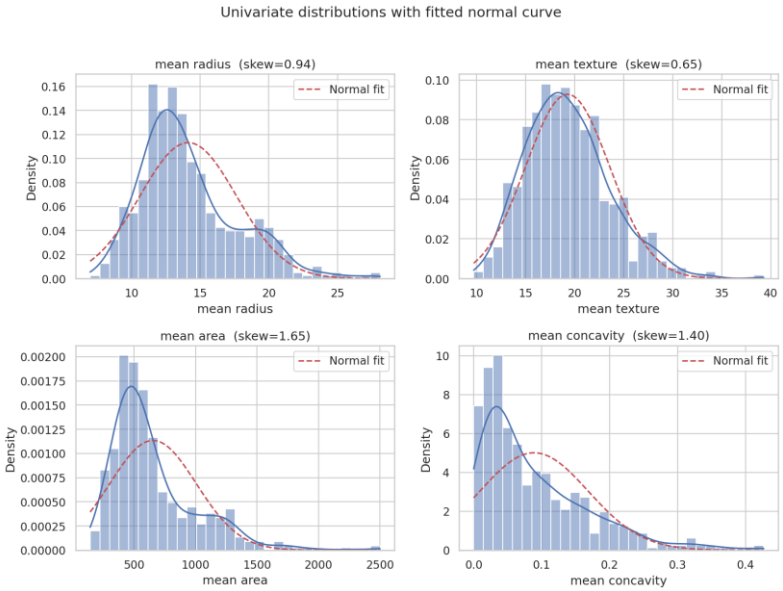


Figure 3: Histograms with fitted normal curve

What it shows: Density histograms with a KDE and a fitted normal curve for mean radius, mean texture, mean area and mean concavity.

Why it was created: To study each continuous variable's distribution (univariate analysis) and compare it with the normal distribution.

Insights obtained: Mean radius and mean texture are roughly bell-shaped (with mild right skew), whereas mean area and mean concavity are clearly right-skewed with long tails, so the normal curve fits them poorly. This supports scaling and outlier control before neural-network training.

Boxplots by diagnosis

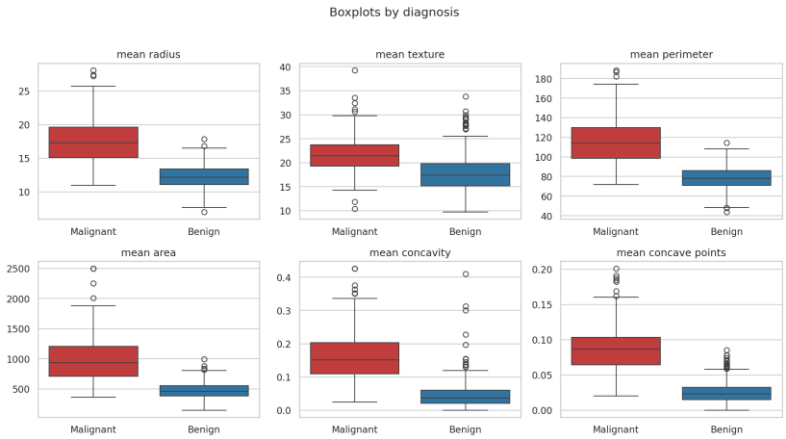


Figure 4: Boxplots by diagnosis

What it shows: Boxplots of six 'mean' features, split by malignant and benign.

Why it was created: To compare class medians and spreads and to spot outliers.

Insights obtained: Malignant tumours have clearly higher radius, perimeter, area, concavity and concave points; the boxes overlap little for concave points and perimeter, which makes these good discriminators. Texture overlaps more. Several points lie beyond the whiskers, i.e., statistical outliers.

Correlation heatmap

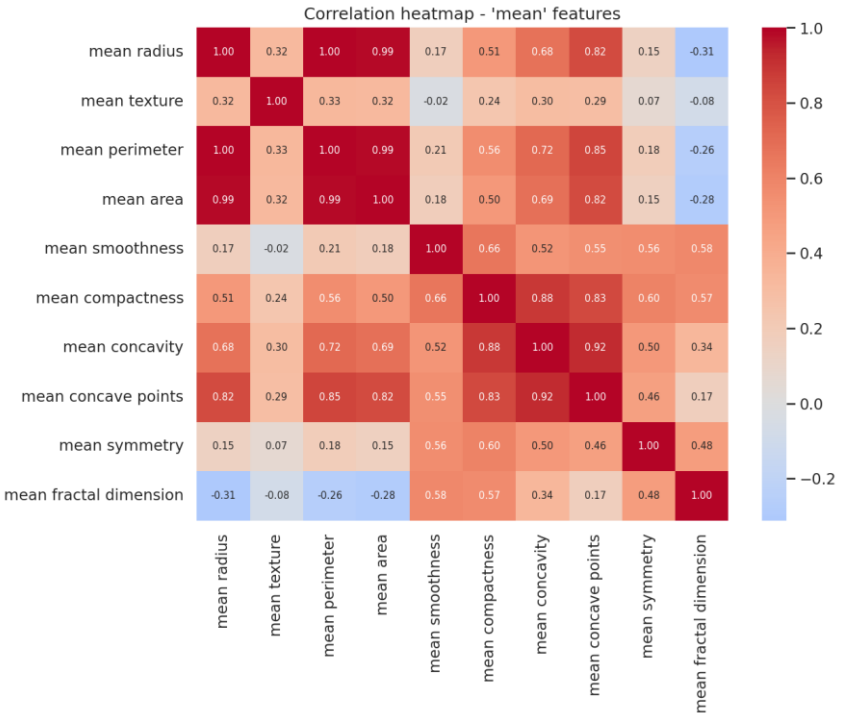


Figure 5: Correlation heatmap

What it shows: Pearson correlation coefficients between the ten 'mean' features.

Why it was created: To detect multicollinearity and relationships among variables (multivariate analysis).

Insights obtained: Radius, perimeter and area are almost perfectly correlated (r above 0.98) because they are geometrically related; concavity, concave points and compactness form a second correlated group. Smoothness and fractal dimension are weakly related to size. Redundant features justify regularization (dropout) in the model.

Scatterplot: radius vs concave points

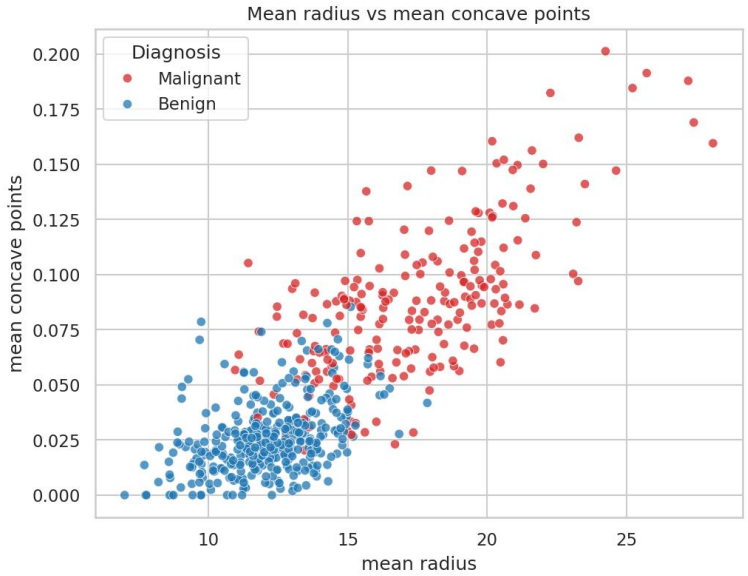


Figure 6: Scatterplot: radius vs concave points

What it shows: Each patient plotted by mean radius (x) and mean concave points (y), coloured by diagnosis.

Why it was created: To visualize bivariate class separation in two of the most discriminating features.

Insights obtained: The two classes form largely separate clusters: malignant cases sit at larger radius and higher concave-points values. Only a narrow overlapping zone exists, which explains why even simple models perform well on this dataset.

Pairplot of the five most informative features

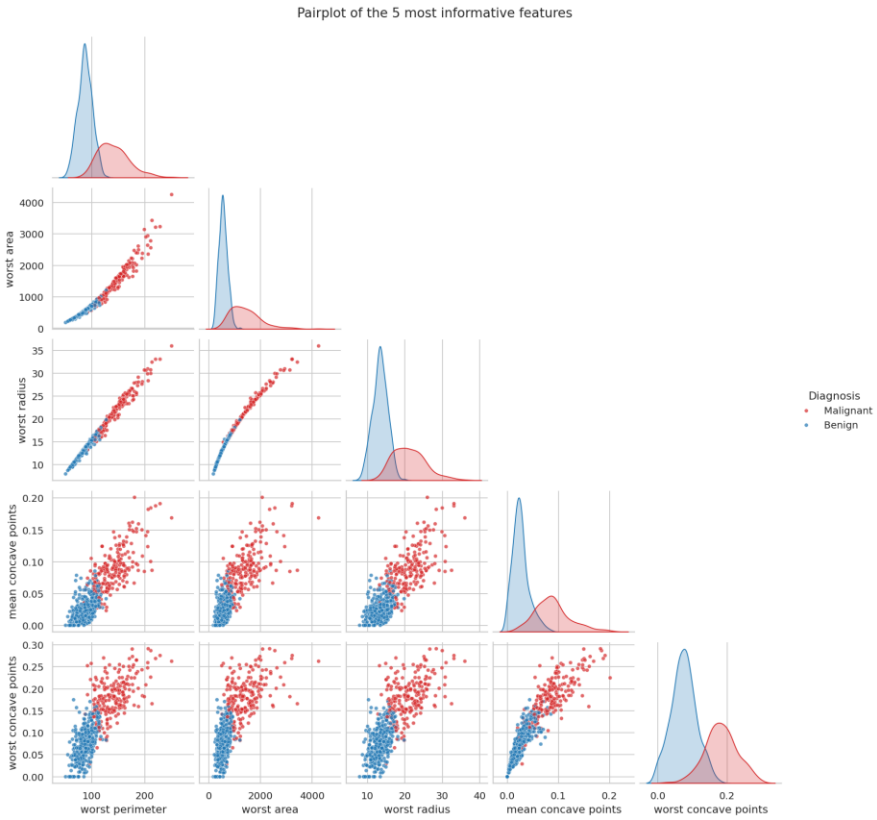


Figure 7: Pairplot of the five most informative features

What it shows: Pairwise scatterplots and diagonal histograms of the top five features ranked by mutual information.

Why it was created: To examine multivariate structure and class separation across several features at once.

Insights obtained: Worst perimeter, worst area and worst radius form near-linear relationships (redundant), and all five features separate the classes along the diagonal distributions.

Q-Q plots

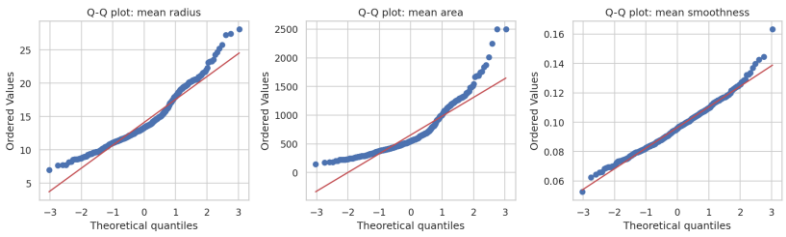


Figure 8: Q-Q plots

What it shows: Quantile-quantile plots of mean radius, mean area and mean smoothness against a normal distribution.

Why it was created: To assess normality of continuous variables more precisely than a histogram.

Insights obtained: Points far from the red line in the upper tail of mean area show a right-skewed (approximately log-normal) distribution, while mean smoothness is closer to normal. Features that deviate from normality benefit from standardization.

Mutual information with the diagnosis

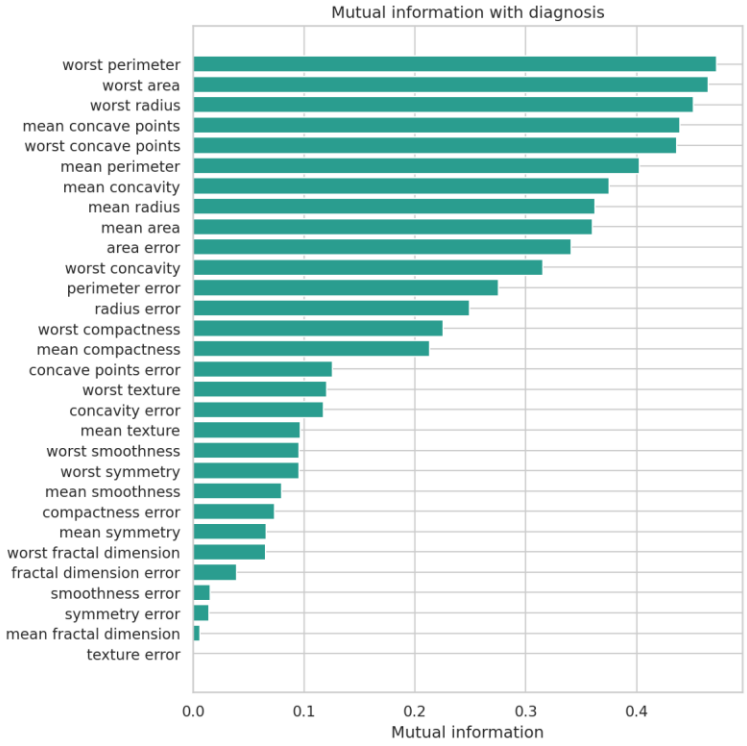


Figure 9: Mutual information with the diagnosis

What it shows: Horizontal bar chart of the mutual information between each feature and the diagnosis.

Why it was created: To rank features by how much information they carry about the target without assuming linearity (feature analysis).

Insights obtained: The top features are worst perimeter, worst area, worst radius, mean concave points, worst concave points. 'Worst' and concave-point features dominate, while several standard-error features and fractal-dimension features carry little information.

Effect of standardization

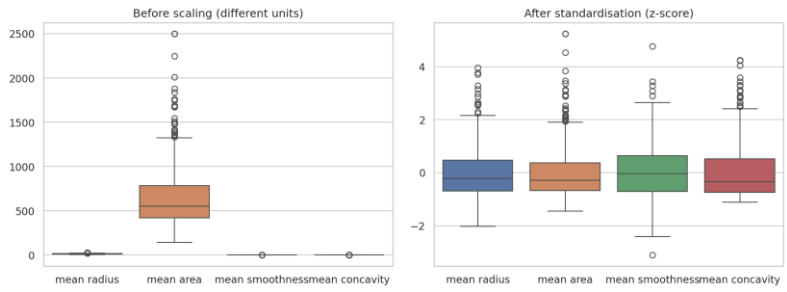


Figure 10: Effect of standardization

What it shows: Boxplots of four features before and after z-score standardization.

Why it was created: To refine the data representation and show why scaling is required (the 'Refine' stage).

Insights obtained: Before scaling, mean area (hundreds to thousands) dwarfs smoothness (about 0.1), which would dominate gradient updates. After standardization all features share a comparable scale.

7.2 Extended statistical visualizations

Outlier count per feature

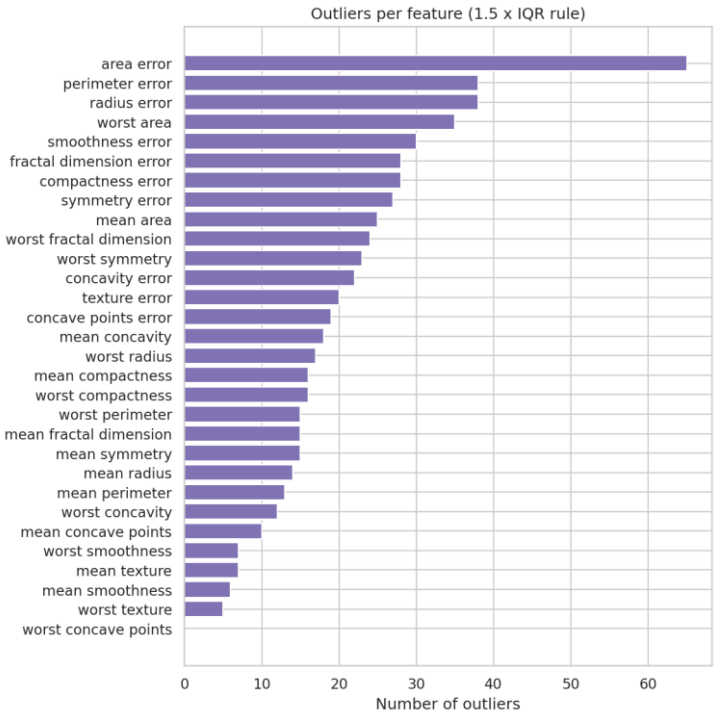


Figure 11: Outlier count per feature

What it shows: A horizontal bar chart of how many observations of each feature fall outside the 1.5 x IQR fences.

Why it was created: To locate which variables contain extreme values and to decide whether they need treatment (Filter stage).

Insights obtained: 29 of 30 features contain outliers (608 values in total). Area error has the most (65), followed by radius error and perimeter error (38 each); four of the five most affected features are standard-error measures. Only 'worst concave points' has no outliers. Since large values reflect large tumours, they were clipped rather than deleted.

Skewness of all features

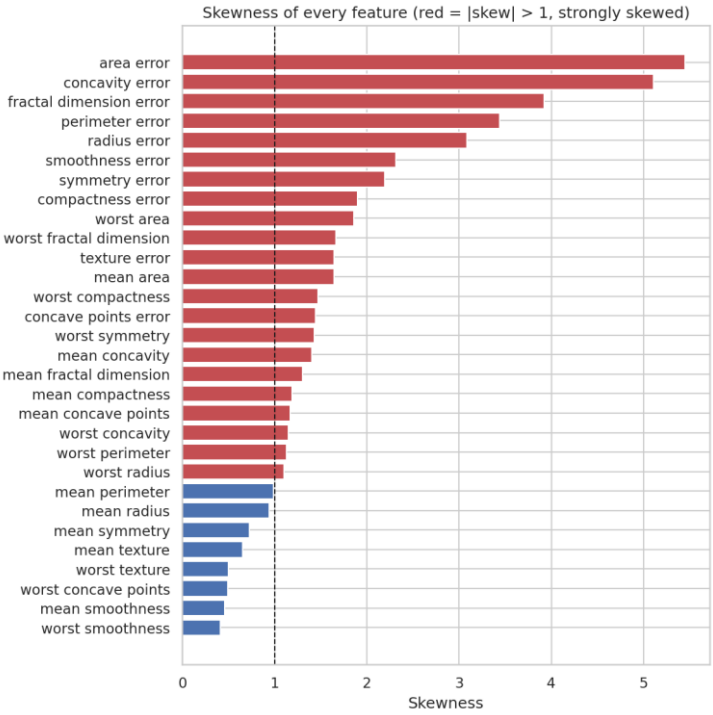


Figure 12: Skewness of all features

What it shows: The skewness coefficient of every feature, with the |skew| = 1 reference line.

Why it was created: To quantify non-normality across all variables at once, which determines whether scaling or transformation is needed.

Insights obtained: 22 features have skewness above 1. The most skewed are area error (5.45), concavity error (5.11), fractal dimension error (3.92); the least skewed are worst smoothness, mean smoothness, worst concave points. Strong right skew suggests that log-transformation or robust scaling could further help, and it explains the poor normal fits seen earlier.

Class-wise density of the 'mean' features

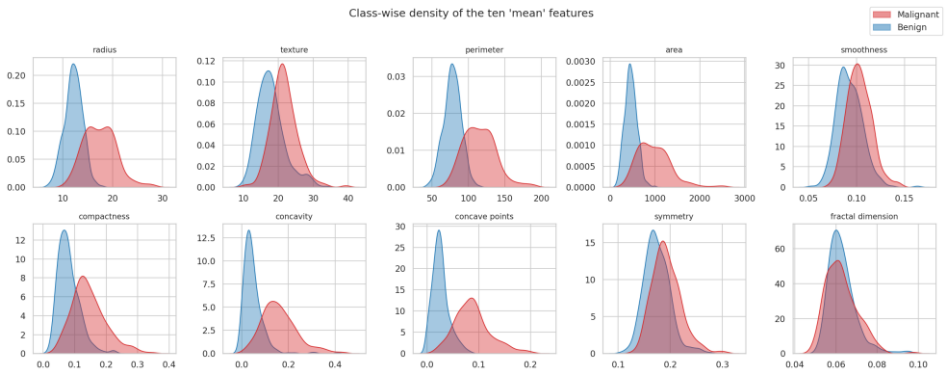


Figure 13: Class-wise density of the 'mean' features

What it shows: Kernel density estimates of the ten mean features, drawn separately for malignant and benign patients.

Why it was created: To judge, feature by feature, how well a single variable separates the two classes (bivariate analysis against the target).

Insights obtained: For radius, perimeter, area, concavity and concave points the malignant density lies clearly to the right with limited overlap. For smoothness (Cohen's d = 0.79) and symmetry (d = 0.72) the separation is weaker than for the size features, and for mean fractal dimension (d = -0.03, Welch p = 0.77) the two curves almost coincide, so it has no discriminating value on its own.

Correlation heatmap of all 30 features

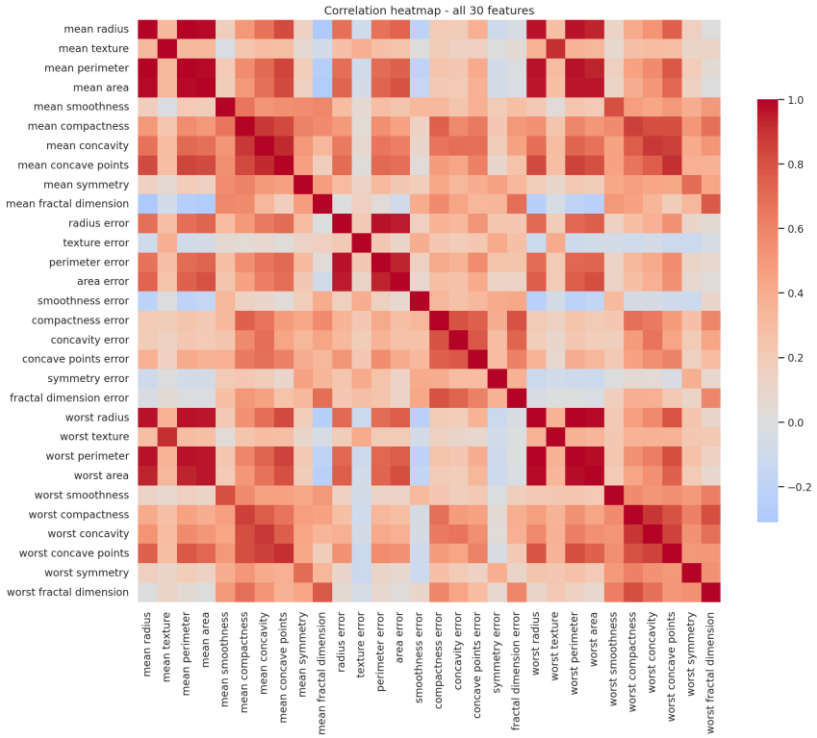


Figure 14: Correlation heatmap of all 30 features

What it shows: The Pearson correlation matrix of all thirty features.

Why it was created: To see the complete redundancy structure, not only that of the mean features (multivariate analysis).

Insights obtained: The heatmap shows bright blocks along the diagonal: the mean and worst versions of the same measurement are strongly related (mean radius vs worst radius r = 0.97), and radius, perimeter and area form one dominant block. 21 feature pairs exceed |r| = 0.9. Standard-error features are generally less correlated with the rest. This multicollinearity does not hurt a neural network's predictions but makes individual feature-importance values unstable.

Correlation with malignancy and effect size

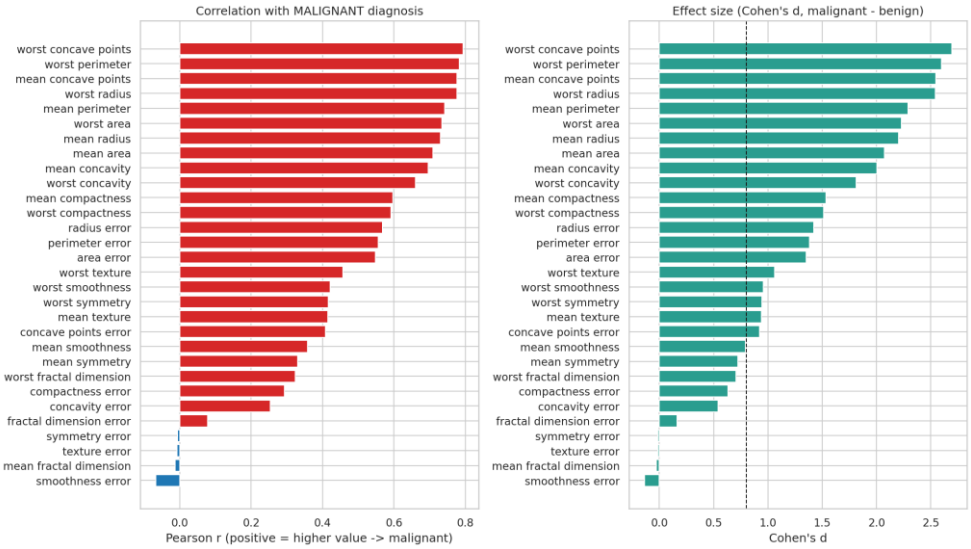


Figure 15: Correlation with malignancy and effect size

What it shows: Left: Pearson correlation of each feature with a 'malignant' indicator. Right: Cohen's d between malignant and benign patients for each feature.

Why it was created: To rank features by how strongly and how consistently they differ between classes, in both correlation and standardized-difference terms.

Insights obtained: Concave-points, perimeter, radius and area features (mean and worst) lead both rankings; the strongest effect is for worst concave points (d = 2.69). 20 features exceed the 'large effect' threshold of 0.8. The features near zero (texture error, mean fractal dimension, symmetry error, smoothness error) carry almost no class information.

Violin plots of the 'worst' features

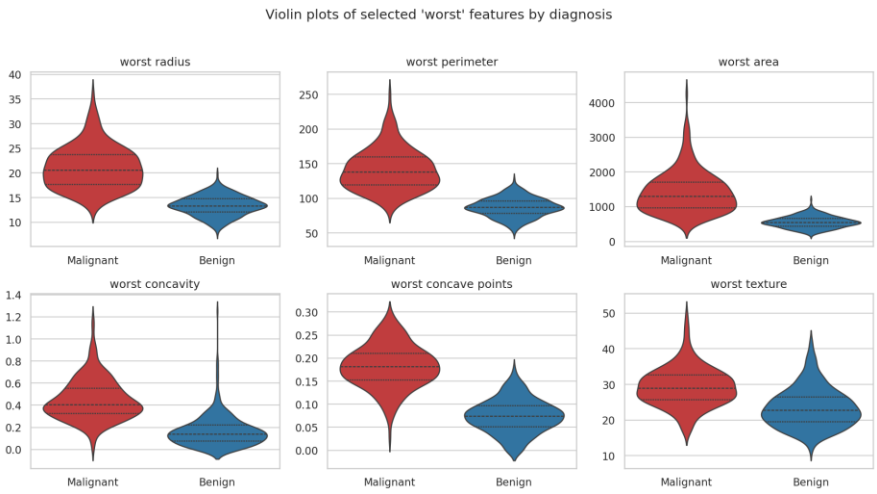


Figure 16: Violin plots of the 'worst' features

What it shows: Violin plots (mirrored density + quartiles) of six 'worst' features, split by diagnosis.

Why it was created: To compare the full shape of each distribution, not only the median and quartiles shown in boxplots.

Insights obtained: Malignant distributions are shifted upwards and are wider than benign ones. For example, the median worst area is 1303 for malignant versus 547 for benign patients. The benign distributions are narrow and compact, whereas malignant ones have long upper tails, which indicates greater variability among malignant nuclei.

Distribution fitting: normal vs log-normal

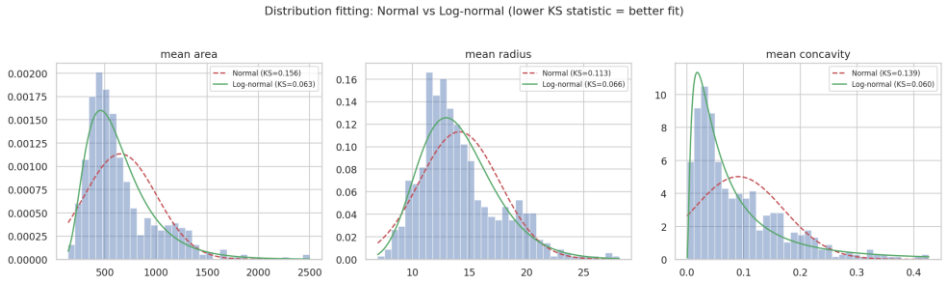


Figure 17: Distribution fitting: normal vs log-normal

What it shows: Histograms of mean area, mean radius and mean concavity with fitted normal and log-normal curves and their KS statistics.

Why it was created: To identify the continuous probability distribution that best describes the skewed features (CO4 requirement).

Insights obtained: The log-normal curve follows the right-skewed histograms much more closely; its KS statistic (area 0.063, radius 0.066, concavity 0.060) is about half that of the normal fit. These measurements are positive and grow multiplicatively, which is typical of log-normal data.

Principal component analysis

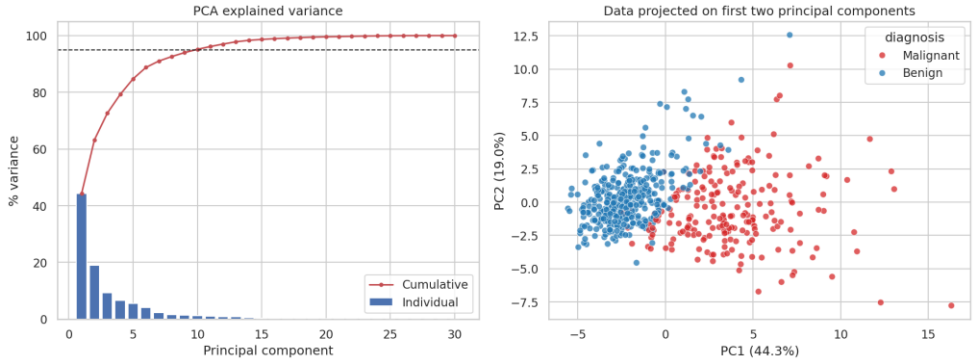


Figure 18: Principal component analysis

What it shows: Left: variance explained by each principal component and the cumulative curve. Right: the patients projected on the first two components.

Why it was created: To measure how many independent dimensions exist in the correlated feature set and to obtain a two-dimensional picture of class separation.

Insights obtained: PC1 explains 44.3% and PC2 19.0% of the variance; 10 components reach 95%. In the 2-D projection the benign patients lie mostly on the left and the malignant on the right, separated mainly along PC1 with a narrow overlap zone. This confirms that the classes are almost linearly separable, which explains the strong performance of even a linear baseline.

7.3 Discrete and continuous probability distributions

- **Discrete:** The diagnosis of each patient is a Bernoulli trial (benign probability about 0.63); the number of benign cases in a sample follows a Binomial distribution (Section 6.4).
- **Continuous:** Mean radius and mean texture are roughly normal (mild right skew); area, concavity and error features are right-skewed and are better modelled by the log-normal distribution (KS comparison above). Normal-curve overlays, Q-Q plots and Shapiro-Wilk tests were used for identification.

7.4 Interactive visualization (stage 7: Interact)

The repository script generates an interactive Plotly scatterplot (results/interactive_scatter.html) with hover tooltips, zoom and legend toggling; bubble size encodes mean area. It is produced when the plotly package is installed (pip install plotly) and is available in the GitHub repository.

8. Deep Learning Model

A Multilayer Perceptron (MLP) was chosen because the data is tabular with numerical features and the classes are nearly separable, so a small fully connected network is sufficient. Convolutional or recurrent networks would add complexity without benefit for 30 independent numeric inputs. Implementation: NumPy MLP (TensorFlow not installed).

8.1 Architecture

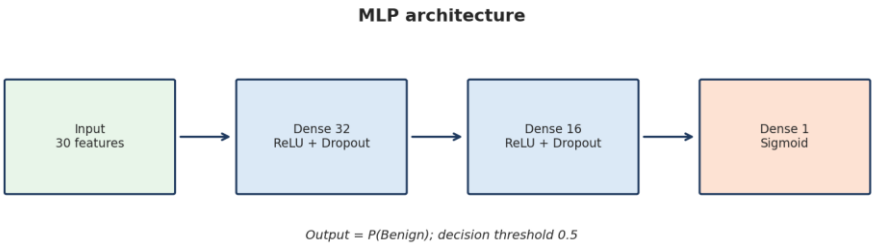


Figure 19: Architecture of the MLP classifier

Table 9: MLP architecture (1,537 trainable parameters)

Layer	Type	Output units	Parameters	Activation / Regularization
Input	Input	30	0	Standardized features
Hidden 1	Dense + Dropout	32	992	ReLU, dropout = 0.2
Hidden 2	Dense + Dropout	16	528	ReLU, dropout = 0.2
Output	Dense	1	17	Sigmoid (probability of Benign)

The network has 1,537 trainable parameters for 398 training samples, which is a small ratio of samples to parameters, so regularization (dropout and early stopping) is important. Weights of the hidden layers are initialised with the He method, appropriate for ReLU activations.

8.2 Training parameters

Table 10: Training parameters

Parameter	Value
Optimizer	Adam (learning rate = 0.01)
Loss	binary cross-entropy
Batch size	32
Maximum epochs	100
Early stopping	monitor val_loss, patience = 15, restore best weights
Epochs actually run / best epoch	30 / 15
Training / validation / test samples	398 / 85 / 86
Random seed	42

8.3 Hyperparameter tuning

A basic grid search over three hidden-layer sizes, two dropout rates and two learning rates (12 configurations) was run. Each model was trained on the training set with early stopping and compared on the validation set; the test set was not used for tuning. The configuration with the lowest validation loss was selected: hidden units (32, 16), dropout 0.2, learning rate 0.01. Because the validation set is small (85 samples), differences of one sample in accuracy are not statistically meaningful, so validation loss (a smoother measure) was used as the criterion.

Table 11: All 12 hyperparameter configurations (sorted by validation loss)

Hidden units	Dropout	Learning rate	Best val. loss	Best val. acc.	Epochs
(32, 16)	0.2	0.01	0.0094	1.0	30
(32, 16)	0.3	0.01	0.0156	0.9882	30
(16, 8)	0.3	0.01	0.0234	1.0	39
(64, 32)	0.3	0.01	0.0303	0.9882	37
(64, 32)	0.3	0.001	0.0341	0.9765	78
(64, 32)	0.2	0.01	0.0342	0.9882	19
(16, 8)	0.2	0.001	0.0344	1.0	78
(64, 32)	0.2	0.001	0.0346	0.9765	74
(16, 8)	0.3	0.001	0.0349	1.0	78
(32, 16)	0.2	0.001	0.0353	0.9765	82
(32, 16)	0.3	0.001	0.0372	0.9765	84
(16, 8)	0.2	0.01	0.0434	1.0	18

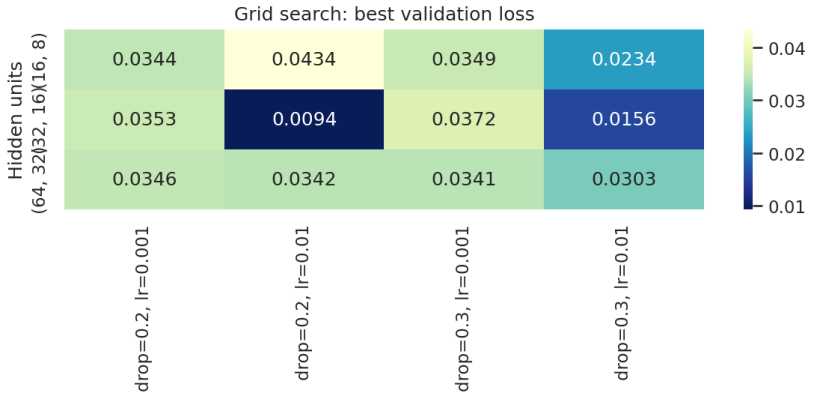


Figure 20: Validation loss of every configuration in the grid search

Averaged over the other settings, learning rate 0.01 gave the lowest validation loss (0.0261 vs 0.0351), and the hidden-layer size (32, 16) had the lowest average loss (0.0244). The single best configuration was (32, 16) units, dropout 0.2, learning rate 0.01. The differences between configurations are small in absolute terms, which is expected for a dataset this size and this well separated.

8.4 Feature analysis and importance

Feature importance was computed in two ways: (i) permutation importance of the trained MLP (the drop in validation accuracy when one feature is shuffled) and (ii) a Random Forest as an independent reference. These complement the mutual-information and effect-size rankings from the EDA.

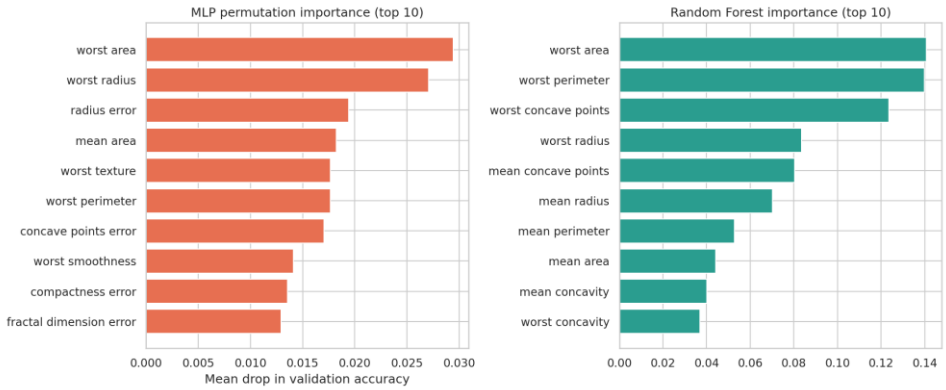


Figure 21: Feature importance - MLP permutation importance and Random Forest importance

Both methods place the size-related 'worst' features at the top (MLP: worst area, worst radius, radius error, mean area, worst texture; Random Forest: worst area, worst perimeter, worst concave points, worst radius, mean concave points), consistent with the mutual-information and Cohen's d rankings. Permutation importance is small and spread out for the MLP because correlated features share the same information - shuffling one is compensated by its correlated partners.

9. Result Visualization and Interpretation

9.1 Learning curves

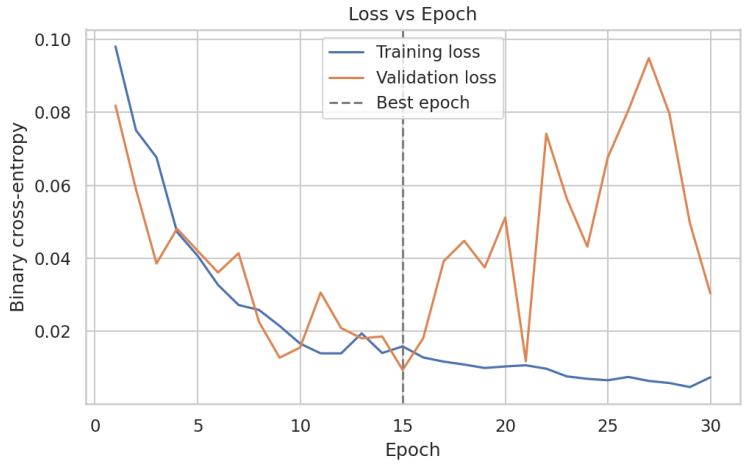


Figure 22: Loss vs Epoch

Training loss decreases steadily as the network learns. The validation loss reaches its minimum at epoch 15 and then stops improving, so early stopping (patience 15) halted training at epoch 30 and restored the weights from epoch 15. The validation curve is noisier than the training curve because the validation set is small (85 samples) and the learning rate is 0.01; early stopping prevents the network from overfitting the training data.

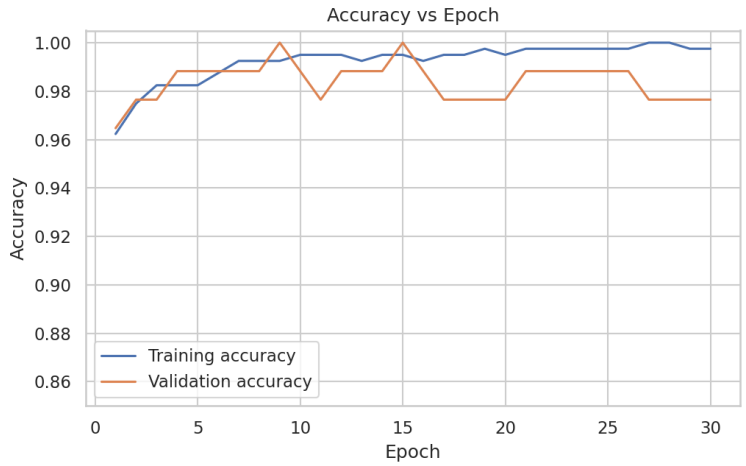


Figure 23: Accuracy vs Epoch

Accuracy at the best epoch is 99.5% (training) and 100.0% (validation). The model learns the task quickly because the classes are well separated (see the radius vs concave-points scatterplot and the PCA projection).

9.2 Test-set performance

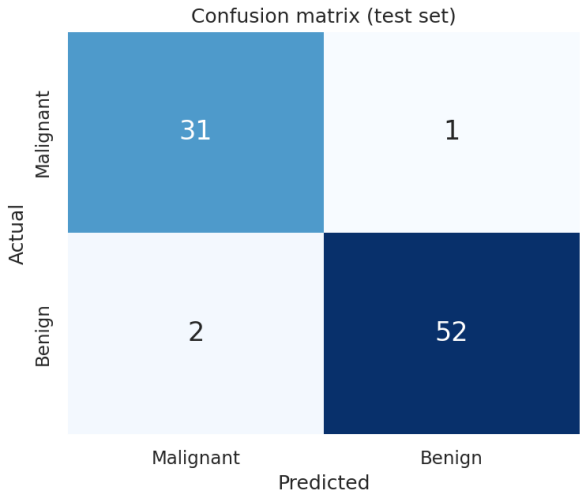


Figure 24: Confusion matrix on the test set

Of 86 test patients, 52 benign and 31 malignant cases were classified correctly; 1 malignant case(s) were wrongly predicted as benign (false positive for the Benign class) and 2 benign case(s) as malignant. In a clinical setting a malignant tumour predicted as benign is the most serious error, so this count should be minimized.

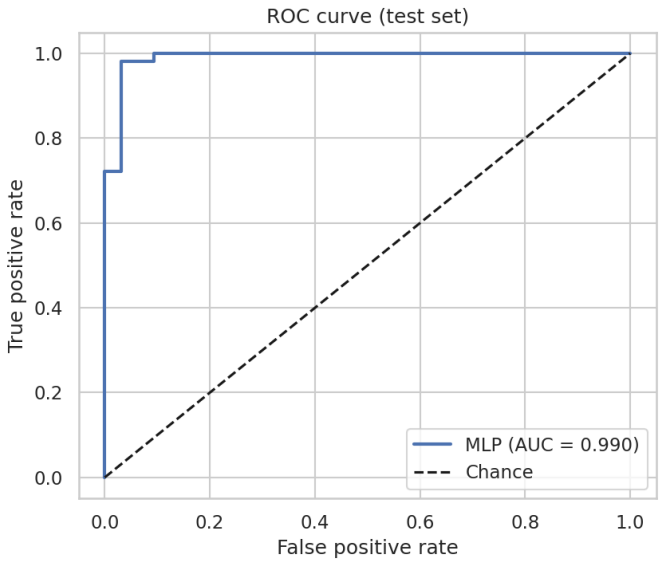


Figure 25: ROC curve and AUC on the test set

The ROC curve hugs the top-left corner and the area under the curve is 0.990, meaning the model ranks a random benign case above a random malignant case with probability 99.0%.

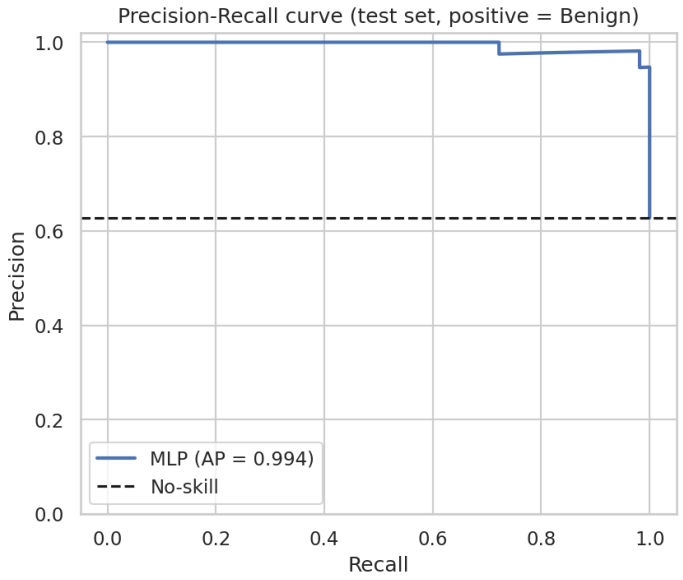


Figure 26: Precision-recall curve on the test set

The precision-recall curve summarises the trade-off between finding all benign cases (recall) and keeping the predictions correct (precision). The average precision is 0.994 (1.0 would be perfect), so the model keeps high precision across most recall levels.

9.3 Metrics summary

Table 12: Test-set performance of the MLP

Metric	Value
Accuracy	0.9651
Precision (Benign)	0.9811
Recall / sensitivity (Benign)	0.9630
Specificity (Malignant correctly found)	0.9688
F1-score	0.9720
ROC-AUC	0.9902
Average precision	0.9935
95% confidence interval for accuracy (normal approx.)	96.5% +/- 3.9 points

Table 13: Per-class classification report (test set)

Class	Precision	Recall	F1-score	Support
Malignant	0.939	0.969	0.954	32
Benign	0.981	0.963	0.972	54
Macro average	0.960	0.966	0.963	86
Weighted average	0.966	0.965	0.965	86

The confidence interval shows how uncertain a single 86-patient test set is: the true accuracy could plausibly lie within about 3.9 percentage points of the measured value.

9.4 Comparison with baseline models

Table 14: Model comparison on the same test set

Model	Test accuracy	Test ROC-AUC
MLP (tuned)	0.9651	0.9902
Logistic regression	0.9884	0.9948
Random forest (300 trees)	0.8953	0.9806

The logistic regression baseline reached 98.8% accuracy, which is slightly higher than the MLP (96.5%). On small, well-separated tabular datasets a simple linear model is often competitive, which shows that a deep model is not automatically superior; its main benefit here is as a learning exercise and as a base for larger data. With only 86 test samples, a difference of one patient changes accuracy by about 1.2 percentage points, so the ranking of models from a single split should not be over-interpreted; cross-validation would be needed for a firm conclusion.

9.5 Threshold and probability analysis

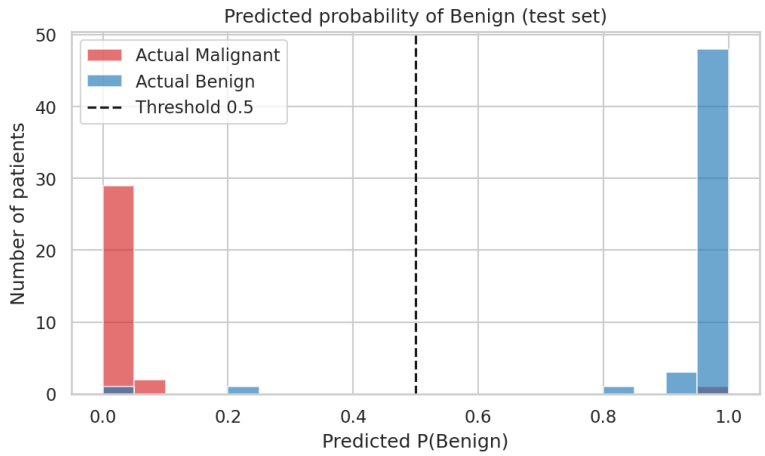


Figure 27: Distribution of predicted probabilities on the test set

98% of the test predictions have a probability below 0.1 or above 0.9, i.e., the network is very confident for the large majority of patients, and only a few cases lie in between. A bimodal distribution like this is typical of an easy, well-separated classification problem.

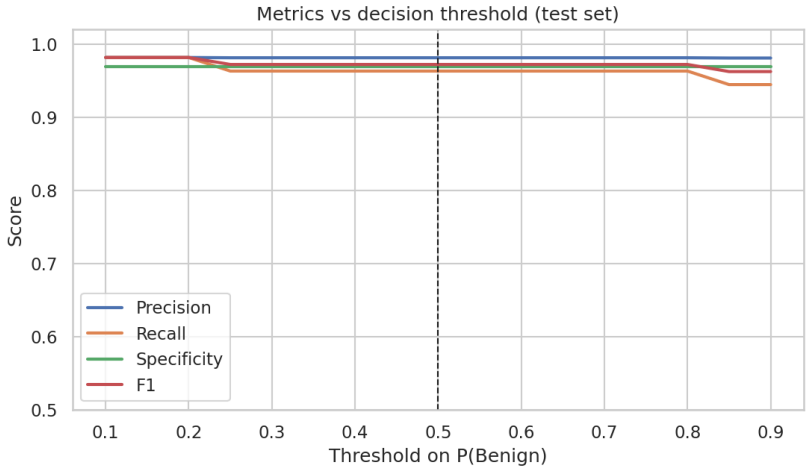


Figure 28: Precision, recall, specificity and F1 against the decision threshold

Table 15: Effect of the decision threshold (full table in results/threshold_analysis.csv)

Threshold	Precision	Recall	Specificity	F1	Accuracy	Malignant missed	Benign flagged
0.1	0.982	0.982	0.969	0.982	0.977	1	1
0.3	0.981	0.963	0.969	0.972	0.965	1	2
0.5	0.981	0.963	0.969	0.972	0.965	1	2
0.7	0.981	0.963	0.969	0.972	0.965	1	2
0.9	0.981	0.944	0.969	0.962	0.954	1	3

The metrics are identical to those at the default threshold 0.5 for every threshold from 0.25 to 0.80, because the predictions are so confident. Outside this band recall or precision changes by only one or two patients. The number of malignant patients missed ranges from 1 to 1 across all thresholds tested (0.10 to 0.90), so for this model threshold adjustment alone cannot remove that error (see the error analysis below).

9.6 Error analysis

The model misclassified 3 of 86 test patients:

Table 16: Misclassified test patients

Test row	Actual	Predicted	P(Benign)
6	Malignant	Benign	0.998
8	Benign	Malignant	0.032
33	Benign	Malignant	0.247

The most serious error is test row 6: a malignant tumour assigned P(Benign) = 0.998, i.e., a highly confident wrong prediction. Such a case may lie in the narrow overlap zone between the classes visible in the scatterplot and PCA projection, or may have atypical measurements; the individual measurements should be inspected to know. It illustrates that model confidence is not proof of correctness and that borderline or surprising cases in practice need review by a pathologist. Errors in the opposite direction (benign tumours flagged as malignant, 2 case(s)) would lead to further tests rather than a missed diagnosis.

10. Discussion

The analysis supports a consistent story. The EDA showed that malignant nuclei are larger and more irregular (larger radius, perimeter, area, concavity and concave points), that these differences are large in standardized terms (Cohen's d above 2.5 for the best features), and that the information is highly redundant (21 feature pairs above r = 0.9, about ten effective dimensions). The same features emerged as the most important for the neural network, the random forest and the mutual-information ranking, which gives confidence that the model learned medically plausible patterns rather than artefacts.

The strong results of every model (96.5% accuracy, AUC 0.990) are a direct consequence of the clear class separation seen in the visualizations. That a simple linear baseline reaches 98.8% against the MLP's 96.5% is consistent with near-linear separability in the PCA projection. For a larger or noisier dataset - or when the raw images are used - a deeper model such as a CNN would be expected to show a clearer advantage.

From an application point of view, the key lesson is that overall accuracy hides the type of error. In cancer screening a false reassurance is far more costly than a false alarm, so a deployed system would be tuned for high sensitivity to malignancy, accompanied by calibrated probabilities and a mandatory human review of uncertain or contradicting cases.

Finally, the study demonstrates the value of EDA before modelling: scaling was needed because of unit differences, outliers were detected and treated without discarding clinically important patients, and redundancy was identified, which justifies regularization and suggests that feature selection could simplify the model without losing accuracy.

11. Limitations and Ethical Considerations

11.1 Limitations

- Sample size: 569 patients is small for deep learning; the test set has only 86 patients, so each error changes accuracy by about 1.2 percentage points and confidence intervals are wide.
- Single split: results come from one stratified split; cross-validation would give more stable estimates and fairer model comparison.
- Single source: the data were collected at one institution and with one imaging protocol, so performance on other hospitals' data is unknown.
- Pre-extracted features: the study uses expert-defined numeric features, not the original images, so it cannot exploit image details that a CNN could learn.
- Missing context: no patient demographics, history or imaging modality information is available, so subgroup performance and fairness cannot be assessed.
- Tuning scope: the grid search was small and based on a small validation set.

11.2 Ethical considerations

The dataset is a public, anonymised research dataset and contains no personal identifiers. Nevertheless, any medical prediction model raises ethical issues: errors have real consequences, models can perform worse on groups under-represented in the training data, and probabilities can be over-trusted. This project is an educational exercise and must not be used for clinical decisions; any real-world deployment would require prospective validation, regulatory approval, bias assessment and clinician oversight.

12. Conclusion and Future Scope

12.1 Conclusion

- **What was learned:** The dataset is clean (no missing values or duplicates), moderately imbalanced, with skewed features and many outliers corresponding to large tumours; about ten principal components capture 95% of its variance.
- **Key patterns:** Size and shape features (radius, perimeter, area, concavity, concave points - especially the 'worst' values) differ strongly between malignant and benign tumours; these features are highly inter-correlated, and log-normal distributions describe the skewed ones better than normal distributions.
- **Model performance:** The tuned MLP reached 96.5% accuracy, 0.972 F1 and 0.990 AUC on unseen test data, compared with 98.8% accuracy for a logistic regression baseline. Of 86 test patients, 3 were misclassified.
- **Challenges and limitations:** Small data size (569 rows) makes results sensitive to the particular split; heavy multicollinearity; the small validation set limits reliable tuning; and the model is not a medical-grade tool.

12.2 Future scope

- Use k-fold cross-validation or repeated splits to obtain confidence intervals instead of a single split.
- Remove redundant features with PCA or correlation filtering and compare model performance.
- Apply a log or Box-Cox transform to the strongly skewed features before scaling.
- Tune hyperparameters with Keras Tuner or Optuna, and compare with gradient boosting (XGBoost / LightGBM).
- Use explainability tools such as SHAP or LIME to explain individual predictions to clinicians.
- Calibrate probabilities and choose a decision threshold that prioritises recall for malignant cases.
- Train a CNN directly on the FNA images (a bigger, external dataset) and validate on data from other hospitals.
- Deploy the model as a simple Streamlit or Flask web app for demonstration, with clear warnings that it is not a diagnostic device.

References

[1] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," in Proc. IS&T/SPIE Int. Symp. Electronic Imaging: Science and Technology, vol. 1905, 1993, pp. 861-870.

[2] D. Dua and C. Graff, "UCI Machine Learning Repository: Breast Cancer Wisconsin (Diagnostic) Data Set," Univ. of California, Irvine, 2019. [Online]. Available: https://archive.ics.uci.edu/ml

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2014.

[5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, pp. 1929-1958, 2014.

[6] B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA, USA: O'Reilly Media, 2007.

[7] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[8] J. W. Tukey, Exploratory Data Analysis. Reading, MA, USA: Addison-Wesley, 1977.

[9] O. L. Mangasarian, W. N. Street, and W. H. Wolberg, "Breast cancer diagnosis and prognosis via linear programming," Oper. Res., vol. 43, no. 4, pp. 570-577, 1995.

[10] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90-95, 2007.

[11] M. L. Waskom, "seaborn: statistical data visualization," J. Open Source Softw., vol. 6, no. 60, p. 3021, 2021.

[12] W. H. Wolberg and O. L. Mangasarian, "Multisurface method of pattern separation for medical diagnosis applied to breast cytology," Proc. Natl. Acad. Sci. USA, vol. 87, no. 23, pp. 9193-9196, 1990.

[13] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed. New York, NY, USA: Springer, 2009.

[14] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org

Appendix A - Mapping of the Report to the Assessment Rubric

Table 17: Rubric coverage

Outcome / criterion	Marks	Where addressed in this report
CO3 - Selection and creation of visualizations	3	Section 7: 17 visualizations of different types chosen by data type
CO3 - Quality and presentation	2	Titles, axis labels, legends and colour-coded classes on every chart
CO3 - Interpretation of visualizations	3	Each chart: what it shows / why created / insights
CO4 - Univariate analysis	4	Sections 5.2, 6.3; histograms, boxplots, skewness, KDE, violin charts
CO4 - Discrete / continuous distributions	4	Sections 6.4 and 7.3; Bernoulli/Binomial, Normal vs Log-normal, Q-Q plots
CO4 - Bivariate / multivariate analysis	4	Correlation heatmaps, scatter, pairplot, Cohen's d, PCA
CO4 - Interpretation of findings	4	Sections 6.6, 7 and 10
CO5 - Real-world problem / application	3	Sections 1 and 11: cytology decision support
CO5 - Application of EDA techniques	3	Section 6: cleaning, outliers, scaling, split, PCA
CO5 - Insights and interpretation	3	Sections 6.6, 7, 9, 10
CO5 - Conclusion and recommendations	3	Sections 11 and 12
CO5 - GitHub repository and documentation	4	Appendix C and the repository README, code, report, dataset link
Enrichment - ML / DL model	optional	Sections 8 and 9: MLP, tuning, baselines, ROC, threshold and error analysis

Appendix B - Glossary

Table 18: Glossary of terms

Term	Meaning
AUC	Area under the ROC curve; probability that a random positive is ranked above a random negative
Cohen's d	Difference between two group means in units of pooled standard deviation
Dropout	Regularization that randomly disables neurons during training
EDA	Exploratory data analysis - summarising and visualizing data before modelling
Early stopping	Stopping training when validation loss stops improving
FNA	Fine-needle aspiration - a procedure to sample cells from a mass
IQR	Interquartile range, Q3 minus Q1
KS statistic	Maximum distance between empirical and fitted cumulative distributions
Log-normal	Distribution of a variable whose logarithm is normally distributed
Multicollinearity	Strong correlation between predictor variables
Mutual information	Measure of dependence between a feature and the target
PCA	Principal component analysis - rotation to uncorrelated axes of maximal variance
Recall (sensitivity)	Share of actual positives that are found
ROC curve	True-positive rate versus false-positive rate over all thresholds
Specificity	Share of actual negatives that are correctly identified
Standardization	Rescaling to zero mean and unit variance

Appendix C - Source Code

The complete project, including dataset, scripts, results and this report, is available at: https://github.com/<your-username>/breast-cancer-eda-mlp

src/config.py

```
"""Central configuration so every script uses the same paths and seeds."""
from pathlib import Path


ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
PROC_DIR = DATA_DIR / "processed"
RESULTS_DIR = ROOT / "results"
FIG_DIR = RESULTS_DIR / "figures"
for _d in (DATA_DIR, PROC_DIR, RESULTS_DIR, FIG_DIR):
    _d.mkdir(parents=True, exist_ok=True)


SEED = 42
RAW_CSV = DATA_DIR / "breast_cancer_wisconsin.csv"
TARGET = "diagnosis"          # 0 = Malignant, 1 = Benign (UCI WDBC coding
used by scikit-learn)
CLASS_NAMES = ["Malignant", "Benign"]
```

src/eda_visualization.py

```
"""
Steps 1-3: Data acquisition, EDA, preprocessing and visualisation.


Maps to the 7 stages of data visualisation (Ben Fry):
  1 Acquire  2 Parse  3 Filter  4 Mine  5 Represent  6 Refine  7 Interact


Run:  python src/eda_visualization.py
"""
import json
import warnings


import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import mutual_info_classif
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


from config import CLASS_NAMES, FIG_DIR, PROC_DIR, RAW_CSV,
RESULTS_DIR, SEED, TARGET


warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", context="notebook")
PALETTE = {"Malignant": "#d62728", "Benign": "#1f77b4"}




def save(fig, name):
    fig.tight_layout()
    fig.savefig(FIG_DIR / name, dpi=150, bbox_inches="tight")
    plt.close(fig)




# ---------------------------------------------------------------- Stage 1: ACQUIRE
def acquire():
    """Load the UCI Breast Cancer Wisconsin (Diagnostic) dataset and cache it as
CSV."""
    ds = load_breast_cancer(as_frame=True)
    df = ds.frame.rename(columns={"target": TARGET})
    df.to_csv(RAW_CSV, index=False)
    return df




# ---------------------------------------------------------------- Stage 2: PARSE
def parse(df):
    """Structure, types, missing values, duplicates, summary statistics."""
    info = {
        "rows": int(df.shape[0]),
        "columns": int(df.shape[1]),
        "n_features": int(df.shape[1] - 1),
        "missing_total": int(df.isna().sum().sum()),
        "duplicates": int(df.duplicated().sum()),
        "class_counts": {CLASS_NAMES[k]: int(v) for k, v in
df[TARGET].value_counts().sort_index().items()},
    }
    df.describe().T.round(3).to_csv(RESULTS_DIR / "summary_statistics.csv")
    return info




# ---------------------------------------------------------------- Stage 3: FILTER
def iqr_outliers(X):
    q1, q3 = X.quantile(.25), X.quantile(.75)
    iqr = q3 - q1
    return ((X < q1 - 1.5 * iqr) | (X > q3 + 1.5 * iqr)).sum()




def filter_data(df):
    """Remove duplicates; report outliers (kept - they are clinically valid)."""
    df = df.drop_duplicates().reset_index(drop=True)
    out = iqr_outliers(df.drop(columns=TARGET))
    return df, out




# ---------------------------------------------------------------- Stage 4: MINE
def mine(df):
    """Statistics: skewness, normality, correlation, mutual information, class t-
tests."""
    X, y = df.drop(columns=TARGET), df[TARGET]
    skew = X.skew().sort_values(ascending=False)
    shapiro = {c: stats.shapiro(X[c])[1] for c in X.columns}
    mi = pd.Series(mutual_info_classif(X, y, random_state=SEED),
index=X.columns).sort_values(ascending=False)
    ttest = {c: stats.ttest_ind(X.loc[y == 0, c], X.loc[y == 1, c], equal_var=False)[1]
for c in X.columns}
    corr = X.corr()
    target_corr = X.apply(lambda s: s.corr(y)).sort_values()
    pairs = corr.where(np.triu(np.ones(corr.shape),
1).astype(bool)).stack().sort_values(ascending=False)
    return {"skew": skew, "shapiro": shapiro, "mi": mi, "ttest": ttest, "corr": corr,
            "target_corr": target_corr, "top_pairs": pairs.head(5)}




# ---------------------------------------------------------------- Stage 5: REPRESENT
def represent(df, m):
    X, y = df.drop(columns=TARGET), df[TARGET]
    labels = y.map(dict(enumerate(CLASS_NAMES)))
    mean_cols = [c for c in X.columns if c.startswith("mean ")]


    # V1 - Class distribution (discrete: Bernoulli / Binomial)
    fig, ax = plt.subplots(1, 2, figsize=(9, 4))
    counts = labels.value_counts()[CLASS_NAMES]
    sns.barplot(x=counts.index, y=counts.values, palette=PALETTE, ax=ax[0])
    for i, v in enumerate(counts.values):
        ax[0].text(i, v + 4, str(v), ha="center", fontweight="bold")
    ax[0].set(title="Class distribution (count)", xlabel="Diagnosis",
ylabel="Number of patients")
    ax[1].pie(counts.values, labels=counts.index, autopct="%1.1f%%",
colors=[PALETTE[c] for c in counts.index], startangle=90)
    ax[1].set_title("Class proportion")
    save(fig, "v1_class_distribution.png")


    # V2 - Histograms + KDE + fitted normal curve (continuous distributions)
    sel = ["mean radius", "mean texture", "mean area", "mean concavity"]
    fig, axes = plt.subplots(2, 2, figsize=(11, 8))
    for ax, c in zip(axes.ravel(), sel):
        sns.histplot(X[c], kde=True, stat="density", bins=30, color="#4c72b0",
ax=ax)
        xs = np.linspace(X[c].min(), X[c].max(), 200)
        ax.plot(xs, stats.norm.pdf(xs, X[c].mean(), X[c].std()), "r--", label="Normal
fit")
        ax.set_title(f"{c}  (skew={X[c].skew():.2f})")
        ax.legend()
```

```
    fig.suptitle("Univariate distributions with fitted normal curve", y=1.02,
fontsize=14)
    save(fig, "v2_histograms_normal_fit.png")


    # V3 - Boxplots by class (outliers + class separation)
    fig, axes = plt.subplots(2, 3, figsize=(13, 7))
    for ax, c in zip(axes.ravel(), ["mean radius", "mean texture", "mean perimeter",
"mean area", "mean concavity", "mean concave points"]):
        sns.boxplot(x=labels, y=X[c], order=CLASS_NAMES, palette=PALETTE,
ax=ax)
        ax.set(title=c, xlabel="", ylabel="")
    fig.suptitle("Boxplots by diagnosis", y=1.02, fontsize=14)
    save(fig, "v3_boxplots_by_class.png")


    # V4 - Correlation heatmap (mean features)
    fig, ax = plt.subplots(figsize=(9, 7.5))
    sns.heatmap(X[mean_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm",
center=0, annot_kws={"size": 7}, ax=ax)
    ax.set_title("Correlation heatmap - 'mean' features")
    save(fig, "v4_correlation_heatmap.png")


    # V5 - Scatterplot radius vs concave points coloured by class
    fig, ax = plt.subplots(figsize=(7, 5.5))
    sns.scatterplot(data=X.assign(Diagnosis=labels), x="mean radius", y="mean
concave points", hue="Diagnosis",
                    palette=PALETTE, alpha=.75, edgecolor="w", ax=ax)
    ax.set_title("Mean radius vs mean concave points")
    save(fig, "v5_scatter_radius_concavepoints.png")


    # V6 - Pairplot of 5 strongest features
    top5 = m["mi"].index[:5].tolist()
    g = sns.pairplot(X[top5].assign(Diagnosis=labels), hue="Diagnosis",
palette=PALETTE, corner=True, plot_kws={"s": 18, "alpha": .7})
    g.figure.suptitle("Pairplot of the 5 most informative features", y=1.02)
    g.figure.savefig(FIG_DIR / "v6_pairplot_top5.png", dpi=130,
bbox_inches="tight")
    plt.close(g.figure)


    # V7 - Q-Q plots (normality check for continuous distributions)
    fig, axes = plt.subplots(1, 3, figsize=(13, 4))
    for ax, c in zip(axes, ["mean radius", "mean area", "mean smoothness"]):
        stats.probplot(X[c], dist="norm", plot=ax)
        ax.set_title(f"Q-Q plot: {c}")
    save(fig, "v7_qq_plots.png")


    # V8 - Mutual information / feature-target correlation (Mine -> Represent)
    fig, ax = plt.subplots(figsize=(8, 8))
    mi = m["mi"].sort_values()
    ax.barh(mi.index, mi.values, color="#2a9d8f")
    ax.set(title="Mutual information with diagnosis", xlabel="Mutual information")
    save(fig, "v8_mutual_information.png")



# ---------------------------------------------------------------- Stage 6: REFINE
def refine(df):
    """Show the effect of preprocessing: before / after scaling + outlier clipping."""
    X = df.drop(columns=TARGET)
    cols = ["mean radius", "mean area", "mean smoothness", "mean concavity"]
    fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
    sns.boxplot(data=X[cols], ax=ax[0]); ax[0].set_title("Before scaling (different
units)")
    Xs = pd.DataFrame(StandardScaler().fit_transform(X),
columns=X.columns)[cols]
    sns.boxplot(data=Xs, ax=ax[1]); ax[1].set_title("After standardisation (z-
score)")
    save(fig, "v9_before_after_scaling.png")



# ---------------------------------------------------------------- Stage 7: INTERACT
def interact(df):
    """Interactive Plotly scatter (hover, zoom, legend toggle). Optional
dependency."""
    try:
        import plotly.express as px
    except ImportError:
        print("plotly not installed -> skipping interactive chart (pip install plotly)")
        return False
    d =
df.assign(Diagnosis=df[TARGET].map(dict(enumerate(CLASS_NAMES))))
    fig = px.scatter(d, x="mean radius", y="mean concave points",
color="Diagnosis", size="mean area",
                     hover_data=["mean texture", "mean perimeter"],
                     color_discrete_map=PALETTE, title="Interactive: radius vs concave
points (bubble = area)")
    fig.write_html(RESULTS_DIR / "interactive_scatter.html")
    return True



# ---------------------------------------------------------------- Preprocess + split
def preprocess_split(df):
    """Clip extreme outliers (3*IQR, fences from TRAIN only), standardise (fit on
TRAIN only), 70/15/15 split."""
    X, y = df.drop(columns=TARGET), df[TARGET]
    X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=.30, stratify=y,
random_state=SEED)
    X_val, X_te, y_val, y_te = train_test_split(X_tmp, y_tmp, test_size=.50,
stratify=y_tmp, random_state=SEED)


    q1, q3 = X_tr.quantile(.25), X_tr.quantile(.75)
    lo, hi = q1 - 3 * (q3 - q1), q3 + 3 * (q3 - q1)
    n_clipped = int(((X_tr < lo) | (X_tr > hi)).sum().sum())
    X_tr, X_val, X_te = (s.clip(lo, hi, axis=1) for s in (X_tr, X_val, X_te))


    sc = StandardScaler().fit(X_tr)
    out = {}
    for name, Xs, ys in (("train", X_tr, y_tr), ("val", X_val, y_val), ("test", X_te,
y_te)):
        arr = pd.DataFrame(sc.transform(Xs), columns=X.columns,
index=Xs.index).assign(**{TARGET: ys.values})
        arr.to_csv(PROC_DIR / f"{name}.csv", index=False)
        out[name] = int(len(arr))
    out["extreme_values_clipped"] = n_clipped
    return out



def main():
    df = acquire()                          # 1 Acquire
    info = parse(df)                        # 2 Parse
    df, out = filter_data(df)               # 3 Filter
    m = mine(df)                            # 4 Mine
    represent(df, m)                        # 5 Represent
    refine(df)                              # 6 Refine
    info["interactive_html"] = interact(df)         # 7 Interact
    info["split_sizes"] = preprocess_split(df)
    info["outliers_iqr_total"] = int(out.sum())
    info["outliers_top5"] =
out.sort_values(ascending=False).head(5).astype(int).to_dict()
    info["skew_top3"] = m["skew"].head(3).round(2).to_dict()
    info["n_nonnormal_shapiro"] = int(sum(p < .05 for p in m["shapiro"].values()))
    info["n_sig_ttest"] = int(sum(p < .05 for p in m["ttest"].values()))
    info["mi_top5"] = m["mi"].head(5).round(3).to_dict()
    info["target_corr_top"] = {"most_negative":
m["target_corr"].head(3).round(3).to_dict(),
                               "most_positive": m["target_corr"].tail(3).round(3).to_dict()}
    info["top_corr_pairs"] = {" | ".join(k): round(float(v), 3) for k, v in
m["top_pairs"].items()}
    (RESULTS_DIR / "eda_info.json").write_text(json.dumps(info, indent=2))
    print(json.dumps(info, indent=2))



if __name__ == "__main__":
    main()
```

src/extra_analysis.py

```
"""
Extended analysis (statistical tables, distribution fitting, PCA, extra charts, error
analysis).
Run AFTER eda_visualization.py and model_mlp.py:   python
src/extra_analysis.py
"""
import json
import warnings


import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from matplotlib.patches import FancyBboxPatch
from scipy import stats
from sklearn.decomposition import PCA
from sklearn.feature_selection import mutual_info_classif
from sklearn.metrics import average_precision_score, precision_recall_curve
from sklearn.preprocessing import StandardScaler
```

```
from config import CLASS_NAMES, FIG_DIR, PROC_DIR, RAW_CSV,
RESULTS_DIR, SEED, TARGET


warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", context="notebook")
PALETTE = {"Malignant": "#d62728", "Benign": "#1f77b4"}




def save(fig, name, dpi=150):
    fig.tight_layout()
    fig.savefig(FIG_DIR / name, dpi=dpi, bbox_inches="tight")
    plt.close(fig)




def box(ax, x, y, w, h, text, color="#dbe9f6"):
    ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.02",
fc=color, ec="#1f3a5f", lw=1.5))
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", fontsize=9)




def arrow(ax, x1, y1, x2, y2):
    ax.annotate("", xy=(x2, y2), xytext=(x1, y1), arrowprops=dict(arrowstyle="->",
lw=1.6, color="#1f3a5f"))




def diagrams():
    # pipeline diagram
    fig, ax = plt.subplots(figsize=(11, 3.6)); ax.axis("off"); ax.set_xlim(0, 11);
ax.set_ylim(0, 3.6)
    steps = ["1. Acquire\nUCI / sklearn\n569 x 31", "2. Parse &\nInspect\ntypes,
stats", "3. Clean\nduplicates,\noutliers", "4. Transform\nscale, split\n70/15/15",
             "5. Visualize\nEDA charts\n& statistics", "6. Model\nMLP + tuning", "7.
Evaluate\nROC, CM,\nloss/acc"]
    for i, t in enumerate(steps):
        x = 0.15 + i * 1.55
        box(ax, x, 1.2, 1.3, 1.2, t, "#dbe9f6" if i < 5 else "#fde2d3")
        if i < len(steps) - 1:
            arrow(ax, x + 1.34, 1.8, x + 1.52, 1.8)
    ax.text(5.5, 3.1, "Project pipeline", ha="center", fontsize=13,
fontweight="bold")
    ax.text(5.5, 0.55, "Preprocessing statistics (clip fences, scaler) are learned from
the TRAINING set only -> no data leakage",
            ha="center", fontsize=9.5, style="italic")
    save(fig, "e1_pipeline_diagram.png")


    # MLP architecture diagram
    m = json.loads((RESULTS_DIR / "metrics.json").read_text())["architecture"]
    sizes = [m["input"], *m["hidden"], 1]
    labels = [f"Input\n{sizes[0]} features", f"Dense {sizes[1]}\nReLU + Dropout",
f"Dense {sizes[2]}\nReLU + Dropout", "Dense 1\nSigmoid"]
    fig, ax = plt.subplots(figsize=(10, 3.2)); ax.axis("off"); ax.set_xlim(0, 10);
ax.set_ylim(0, 3.2)
    for i, t in enumerate(labels):
        x = 0.3 + i * 2.5
        box(ax, x, 0.9, 1.8, 1.3, t, ["#e8f5e9", "#dbe9f6", "#dbe9f6", "#fde2d3"][i])
        if i < 3:
            arrow(ax, x + 1.85, 1.55, x + 2.45, 1.55)
    ax.text(5, 2.9, "MLP architecture", ha="center", fontsize=13,
fontweight="bold")
    ax.text(5, 0.35, "Output = P(Benign); decision threshold 0.5", ha="center",
fontsize=9.5, style="italic")
    save(fig, "e2_mlp_architecture.png")




def main():
    df = pd.read_csv(RAW_CSV)
    X, y = df.drop(columns=TARGET), df[TARGET]
    labels = y.map(dict(enumerate(CLASS_NAMES)))
    diagrams()


    # ---------------- per-feature statistics table
    q1, q3 = X.quantile(.25), X.quantile(.75)
    iqr = q3 - q1
    n_out = ((X < q1 - 1.5 * iqr) | (X > q3 + 1.5 * iqr)).sum()
    mal, ben = X[y == 0], X[y == 1]
    pooled = np.sqrt(((len(mal) - 1) * mal.var() + (len(ben) - 1) * ben.var()) /
(len(X) - 2))
    cohen = (mal.mean() - ben.mean()) / pooled
    mi = pd.Series(mutual_info_classif(X, y, random_state=SEED),
index=X.columns)
    tab = pd.DataFrame({
        "mean": X.mean(), "std": X.std(), "skew": X.skew(), "kurtosis": X.kurt(),
        "outliers_iqr": n_out, "shapiro_p": [stats.shapiro(X[c])[1] for c in X.columns],
        "welch_p": [stats.ttest_ind(mal[c], ben[c], equal_var=False)[1] for c in
X.columns],
        "cohen_d": cohen, "mutual_info": mi, "corr_target": X.apply(lambda s:
s.corr(y))})
    tab.round(4).to_csv(RESULTS_DIR / "feature_statistics.csv")


    # group means (mean-features)
    mean_cols = [c for c in X.columns if c.startswith("mean ")]
    grp = pd.DataFrame({"malignant_mean": mal[mean_cols].mean(),
"benign_mean": ben[mean_cols].mean(),
                        "malignant_median": mal[mean_cols].median(), "benign_median":
ben[mean_cols].median()})
    grp["ratio_mal_to_ben"] = grp.malignant_mean / grp.benign_mean
    grp.round(4).to_csv(RESULTS_DIR / "group_means.csv")


    # E3 outliers per feature
    fig, ax = plt.subplots(figsize=(8, 8))
    s = n_out.sort_values()
    ax.barh(s.index, s.values, color="#8172b3"); ax.set(title="Outliers per feature
(1.5 x IQR rule)", xlabel="Number of outliers")
    save(fig, "e3_outliers_per_feature.png")


    # E4 skewness
    fig, ax = plt.subplots(figsize=(8, 8))
    s = X.skew().sort_values()
    ax.barh(s.index, s.values, color=["#c44e52" if v > 1 else "#4c72b0" for v in
s.values])
    ax.axvline(1, color="k", ls="--", lw=1); ax.axvline(0, color="k", lw=1)
    ax.set(title="Skewness of every feature (red = |skew| > 1, strongly skewed)",
xlabel="Skewness")
    save(fig, "e4_skewness.png")


    # E5 KDE by class
    fig, axes = plt.subplots(2, 5, figsize=(17, 6.5))
    for ax, c in zip(axes.ravel(), mean_cols):
        sns.kdeplot(data=X.assign(D=labels), x=c, hue="D", palette=PALETTE,
fill=True, common_norm=False, alpha=.4, ax=ax, legend=False)
        ax.set_title(c.replace("mean ", ""), fontsize=10); ax.set(xlabel="", ylabel="")
    fig.legend(handles=[plt.Rectangle((0, 0), 1, 1, color=PALETTE[k], alpha=.5)
for k in CLASS_NAMES], labels=CLASS_NAMES, loc="upper right")
    fig.suptitle("Class-wise density of the ten 'mean' features", fontsize=14, y=1.02)
    save(fig, "e5_kde_by_class.png")


    # E6 full correlation heatmap
    fig, ax = plt.subplots(figsize=(13, 11))
    sns.heatmap(X.corr(), cmap="coolwarm", center=0, square=True,
cbar_kws={"shrink": .7}, ax=ax)
    ax.set_title("Correlation heatmap - all 30 features")
    save(fig, "e6_full_correlation_heatmap.png")
    corr = X.corr().abs()
    high_pairs = int(((corr.where(np.triu(np.ones(corr.shape), 1).astype(bool))) >
0.9).sum().sum())


    # E7 signed correlation with malignancy + effect size
    fig, ax = plt.subplots(1, 2, figsize=(14, 8))
    c = (-tab["corr_target"]).sort_values()
    ax[0].barh(c.index, c.values, color=["#d62728" if v > 0 else "#1f77b4" for v in
c.values])
    ax[0].set(title="Correlation with MALIGNANT diagnosis", xlabel="Pearson r
(positive = higher value -> malignant)")
    d = tab["cohen_d"].sort_values()
    ax[1].barh(d.index, d.values, color="#2a9d8f"); ax[1].set(title="Effect size
(Cohen's d, malignant - benign)", xlabel="Cohen's d")
    ax[1].axvline(0.8, color="k", ls="--", lw=1)
    save(fig, "e7_target_correlation_effect_size.png")


    # E8 violin of 'worst' features
    wcols = ["worst radius", "worst perimeter", "worst area", "worst concavity",
"worst concave points", "worst texture"]
    fig, axes = plt.subplots(2, 3, figsize=(13, 7))
    for ax, c in zip(axes.ravel(), wcols):
        sns.violinplot(x=labels, y=X[c], order=CLASS_NAMES, palette=PALETTE,
inner="quartile", ax=ax)
        ax.set(title=c, xlabel="", ylabel="")
    fig.suptitle("Violin plots of selected 'worst' features by diagnosis", y=1.02,
fontsize=14)
    save(fig, "e8_violin_worst.png")


    # E9 distribution fitting (normal vs log-normal)
    fits = {}
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.3))
    for ax, c in zip(axes, ["mean area", "mean radius", "mean concavity"]):
        v = X[c].values if c != "mean concavity" else X[c][X[c] > 0].values
        mu, sd = v.mean(), v.std(ddof=1)
        shape, loc, scale = stats.lognorm.fit(v, floc=0)
```

```
        ks_n = stats.kstest(v, "norm", args=(mu, sd)).statistic
        ks_l = stats.kstest(v, "lognorm", args=(shape, loc, scale)).statistic
        fits[c] = {"ks_normal": round(float(ks_n), 4), "ks_lognormal":
round(float(ks_l), 4)}
        xs = np.linspace(v.min(), v.max(), 300)
        ax.hist(v, bins=35, density=True, alpha=.45, color="#4c72b0")
        ax.plot(xs, stats.norm.pdf(xs, mu, sd), "r--", label=f"Normal
(KS={ks_n:.3f})")
        ax.plot(xs, stats.lognorm.pdf(xs, shape, loc, scale), "g-", label=f"Log-normal
(KS={ks_l:.3f})")
        ax.set_title(c); ax.legend(fontsize=8)
    fig.suptitle("Distribution fitting: Normal vs Log-normal (lower KS statistic =
better fit)", y=1.03, fontsize=13)
    save(fig, "e9_distribution_fitting.png")


    # E10 PCA
    Z = StandardScaler().fit_transform(X)
    pca = PCA().fit(Z)
    cum = np.cumsum(pca.explained_variance_ratio_)
    n95 = int(np.searchsorted(cum, .95) + 1)
    pcs = PCA(2).fit_transform(Z)
    fig, ax = plt.subplots(1, 2, figsize=(13, 5))
    ax[0].bar(range(1, 31), pca.explained_variance_ratio_ * 100, color="#4c72b0",
label="Individual")
    ax[0].plot(range(1, 31), cum * 100, "r-o", ms=3, label="Cumulative");
ax[0].axhline(95, color="k", ls="--", lw=1)
    ax[0].set(title="PCA explained variance", xlabel="Principal component",
ylabel="% variance"); ax[0].legend()
    sns.scatterplot(x=pcs[:, 0], y=pcs[:, 1], hue=labels, palette=PALETTE,
alpha=.75, edgecolor="w", ax=ax[1])
    ax[1].set(title="Data projected on first two principal components", xlabel=f"PC1
({pca.explained_variance_ratio_[0]*100:.1f}%)",
              ylabel=f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
    save(fig, "e10_pca.png")


    # E11 hyper-parameter heat-map
    t = pd.read_csv(RESULTS_DIR / "hyperparameter_tuning.csv")
    t["cfg"] = "drop=" + t.dropout.astype(str) + ", lr=" + t.learning_rate.astype(str)
    pv = t.pivot(index="hidden_units", columns="cfg", values="best_val_loss")
    fig, ax = plt.subplots(figsize=(8, 3.8))
    sns.heatmap(pv, annot=True, fmt=".4f", cmap="YlGnBu_r", ax=ax);
ax.set(title="Grid search: best validation loss", xlabel="", ylabel="Hidden units")
    save(fig, "e11_tuning_heatmap.png")


    # E12-E14 model diagnostics from saved test predictions
    pr = pd.read_csv(RESULTS_DIR / "test_predictions.csv")
    yt, p = pr.y_true.values, pr.prob_benign.values
    prec, rec, _ = precision_recall_curve(yt, p)
    ap = average_precision_score(yt, p)
    fig, ax = plt.subplots(figsize=(5.8, 5))
    ax.plot(rec, prec, lw=2, label=f"MLP (AP = {ap:.3f})"); ax.axhline(yt.mean(),
color="k", ls="--", label="No-skill")
    ax.set(title="Precision-Recall curve (test set, positive = Benign)",
xlabel="Recall", ylabel="Precision", ylim=(0, 1.02)); ax.legend(loc="lower left")
    save(fig, "e12_precision_recall.png")


    fig, ax = plt.subplots(figsize=(7, 4.3))
    for k, name in ((0, "Malignant"), (1, "Benign")):
        ax.hist(p[yt == k], bins=np.linspace(0, 1, 21), alpha=.65,
color=PALETTE[name], label=f"Actual {name}")
    ax.axvline(.5, color="k", ls="--", label="Threshold 0.5")
    ax.set(title="Predicted probability of Benign (test set)", xlabel="Predicted
P(Benign)", ylabel="Number of patients"); ax.legend()
    save(fig, "e13_probability_distribution.png")


    rows = []
    for th in np.arange(0.1, 0.95, 0.05):
        pred = (p > th).astype(int)
        tp = int(((pred == 1) & (yt == 1)).sum()); fp = int(((pred == 1) & (yt ==
0)).sum())
        fn = int(((pred == 0) & (yt == 1)).sum()); tn = int(((pred == 0) & (yt ==
0)).sum())
        pr_ = tp / (tp + fp) if tp + fp else 1.0; rc = tp / (tp + fn); sp = tn / (tn + fp)
        rows.append({"threshold": round(float(th), 2), "precision": pr_, "recall": rc,
"specificity": sp,
                     "f1": 2 * pr_ * rc / (pr_ + rc) if pr_ + rc else 0, "accuracy": (tp + tn) /
len(yt),
                     "missed_malignant": fp, "missed_benign": fn})
    th = pd.DataFrame(rows); th.round(4).to_csv(RESULTS_DIR /
"threshold_analysis.csv", index=False)
    fig, ax = plt.subplots(figsize=(7.5, 4.5))
    for c in ("precision", "recall", "specificity", "f1"):
        ax.plot(th.threshold, th[c], label=c.capitalize(), lw=2)
    ax.axvline(.5, color="k", ls="--", lw=1); ax.set(title="Metrics vs decision
threshold (test set)", xlabel="Threshold on P(Benign)", ylabel="Score", ylim=(.5,
1.02)); ax.legend()
    save(fig, "e14_threshold_analysis.png")

    # misclassified test cases
    wrong = pr[pr.y_true != pr.y_pred].copy()
    wrong["actual"] = wrong.y_true.map(dict(enumerate(CLASS_NAMES)));
wrong["predicted"] = wrong.y_pred.map(dict(enumerate(CLASS_NAMES)))
    wrong.to_csv(RESULTS_DIR / "misclassified_test_cases.csv", index=False)


    # binomial example for the test set
    p_b = float((y == 1).mean()); n_t = len(yt)
    binom = {"p_benign": round(p_b, 4), "n": n_t, "expected_benign": round(n_t *
p_b, 1),
             "sd": round(float(np.sqrt(n_t * p_b * (1 - p_b))), 2),
             "p_at_least_54": round(float(stats.binom.sf(53, n_t, p_b)), 4)}


    info = {"high_corr_pairs_gt_0.9": high_pairs, "pca_components_for_95pct":
n95,
            "pca_pc1_pct": round(float(pca.explained_variance_ratio_[0] * 100), 1),
            "pca_pc2_pct": round(float(pca.explained_variance_ratio_[1] * 100), 1),
            "pca_first2_cum_pct": round(float(cum[1] * 100), 1),
"pca_first10_cum_pct": round(float(cum[9] * 100), 1),
            "ks_fits": fits, "binomial": binom, "average_precision": round(float(ap), 4),
            "n_strong_effect_d_gt_0.8": int((tab.cohen_d.abs() > 0.8).sum()),
            "top_effect":
tab.cohen_d.abs().sort_values(ascending=False).head(3).round(2).to_dict(),
            "n_strong_skew": int((tab["skew"].abs() > 1).sum()), "total_outliers":
int(n_out.sum()),
            "features_with_outliers": int((n_out > 0).sum()),
            "median_ratio_area": round(float(grp.loc["mean area",
"ratio_mal_to_ben"]), 2),
            "median_ratio_concave": round(float(grp.loc["mean concave points",
"ratio_mal_to_ben"]), 2),
            "lowest_welch_features": tab[tab.welch_p >= .05].index.tolist(),
"n_misclassified": int(len(wrong))}
    (RESULTS_DIR / "extra_info.json").write_text(json.dumps(info, indent=2))
    print(json.dumps(info, indent=2))



if __name__ == "__main__":
    main()
```

src/numpy_mlp.py

```
"""
Minimal NumPy MLP (ReLU hidden layers, sigmoid output, dropout, Adam, early
stopping).


It mirrors the Keras model in `model_mlp.py` and is used ONLY as a fallback
when
TensorFlow is not installed, so the whole pipeline still runs end-to-end.
"""
import numpy as np



class NumpyMLP:
    def __init__(self, n_in, hidden=(32, 16), dropout=0.2, lr=1e-3, seed=42):
        self.rng = np.random.default_rng(seed)
        sizes = [n_in, *hidden, 1]
        # He initialisation
        self.W = [self.rng.normal(0, np.sqrt(2 / sizes[i]), (sizes[i], sizes[i + 1])) for i
in range(len(sizes) - 1)]
        self.b = [np.zeros(sizes[i + 1]) for i in range(len(sizes) - 1)]
        self.dropout, self.lr = dropout, lr
        self.m = [np.zeros_like(p) for p in self.W + self.b]
        self.v = [np.zeros_like(p) for p in self.W + self.b]
        self.t = 0


    @staticmethod
    def _sigmoid(z):
        return 1 / (1 + np.exp(-np.clip(z, -50, 50)))


    def _forward(self, X, train):
        acts, masks = [X], []
        for i in range(len(self.W) - 1):
            a = np.maximum(0, acts[-1] @ self.W[i] + self.b[i])
            if train and self.dropout > 0:
                mask = (self.rng.random(a.shape) > self.dropout) / (1 - self.dropout)
                a = a * mask
                masks.append(mask)
            else:
                masks.append(None)
            acts.append(a)
        out = self._sigmoid(acts[-1] @ self.W[-1] + self.b[-1]).ravel()
        return out, acts, masks
```

```
    def predict_proba(self, X):
        return self._forward(X, train=False)[0]

    @staticmethod
    def _bce(y, p):
        p = np.clip(p, 1e-7, 1 - 1e-7)
        return float(-np.mean(y * np.log(p) + (1 - y) * np.log(1 - p)))

    def _step(self, Xb, yb):
        p, acts, masks = self._forward(Xb, train=True)
        delta = ((p - yb) / len(yb)).reshape(-1, 1)
        gW, gb = [None] * len(self.W), [None] * len(self.b)
        for i in reversed(range(len(self.W))):
            gW[i] = acts[i].T @ delta
            gb[i] = delta.sum(0)
            if i > 0:
                delta = (delta @ self.W[i].T) * (acts[i] > 0)
                if masks[i - 1] is not None:
                    delta = delta * masks[i - 1]
        self.t += 1
        for k, (p_, g) in enumerate(zip(self.W + self.b, gW + gb)):   # Adam update
(in place)
            self.m[k] = 0.9 * self.m[k] + 0.1 * g
            self.v[k] = 0.999 * self.v[k] + 0.001 * g * g
            mh = self.m[k] / (1 - 0.9 ** self.t)
            vh = self.v[k] / (1 - 0.999 ** self.t)
            p_ -= self.lr * mh / (np.sqrt(vh) + 1e-7)

    def fit(self, Xtr, ytr, Xval, yval, epochs=100, batch_size=32, patience=15):
        hist = {"loss": [], "val_loss": [], "accuracy": [], "val_accuracy": []}
        best, best_w, wait = np.inf, None, 0
        for _ in range(epochs):
            idx = self.rng.permutation(len(ytr))
            for s in range(0, len(idx), batch_size):
                b = idx[s:s + batch_size]
                self._step(Xtr[b], ytr[b])
            ptr, pval = self.predict_proba(Xtr), self.predict_proba(Xval)
            hist["loss"].append(self._bce(ytr, ptr));
hist["val_loss"].append(self._bce(yval, pval))
            hist["accuracy"].append(float(np.mean((ptr > .5) == ytr)))
            hist["val_accuracy"].append(float(np.mean((pval > .5) == yval)))
            if hist["val_loss"][-1] < best - 1e-5:
                best, wait = hist["val_loss"][-1], 0
                best_w = ([w.copy() for w in self.W], [b_.copy() for b_ in self.b])
            else:
                wait += 1
                if wait >= patience:
                    break
        if best_w:
            self.W, self.b = best_w
        return hist
```

src/model_mlp.py

```
"""
Steps 4-5: MLP model (Keras/TensorFlow), basic hyper-parameter tuning,
feature importance and evaluation visualisations.

Run:  python src/model_mlp.py          (after src/eda_visualization.py)

If TensorFlow is not installed, an equivalent NumPy MLP (numpy_mlp.py) is used
and
the backend name is stored in results/metrics.json.
"""
import itertools
import json
import random
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, auc, classification_report,
confusion_matrix, f1_score,
                             precision_score, recall_score, roc_curve)

from config import CLASS_NAMES, FIG_DIR, PROC_DIR, RESULTS_DIR,
SEED, TARGET
from numpy_mlp import NumpyMLP

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
np.random.seed(SEED); random.seed(SEED)

try:                                # ---- backend selection
    import tensorflow as tf
    from tensorflow import keras
    tf.random.set_seed(SEED)
    BACKEND = f"Keras/TensorFlow {tf.__version__}"
except ImportError:
    BACKEND = "NumPy MLP (TensorFlow not installed)"

EPOCHS, BATCH, PATIENCE = 100, 32, 15


def load_data():
    tr, va, te = (pd.read_csv(PROC_DIR / f"{n}.csv") for n in ("train", "val", "test"))
    feats = [c for c in tr.columns if c != TARGET]
    return (tr[feats].values, tr[TARGET].values.astype(float), va[feats].values,
va[TARGET].values.astype(float),
            te[feats].values, te[TARGET].values.astype(float), feats)


class Model:
    """Thin wrapper giving Keras and NumPy models the same interface."""

    def __init__(self, n_in, hidden, dropout, lr):
        self.hidden, self.dropout, self.lr = hidden, dropout, lr
        if BACKEND.startswith("Keras"):
            keras.backend.clear_session()
            layers = [keras.layers.Input((n_in,))]
            for h in hidden:                # Dense -> ReLU -> Dropout blocks
                layers += [keras.layers.Dense(h, activation="relu"),
keras.layers.Dropout(dropout)]
            layers.append(keras.layers.Dense(1, activation="sigmoid"))
            self.m = keras.Sequential(layers)
            self.m.compile(optimizer=keras.optimizers.Adam(lr),
loss="binary_crossentropy", metrics=["accuracy"])
        else:
            self.m = NumpyMLP(n_in, hidden, dropout, lr, SEED)

    def fit(self, Xtr, ytr, Xva, yva):
        if BACKEND.startswith("Keras"):
            es = keras.callbacks.EarlyStopping(monitor="val_loss",
patience=PATIENCE, restore_best_weights=True)
            h = self.m.fit(Xtr, ytr, validation_data=(Xva, yva), epochs=EPOCHS,
batch_size=BATCH, callbacks=[es], verbose=0)
            return {k: [float(x) for x in v] for k, v in h.history.items()}
        return self.m.fit(Xtr, ytr, Xva, yva, EPOCHS, BATCH, PATIENCE)

    def predict_proba(self, X):
        if BACKEND.startswith("Keras"):
            return self.m.predict(X, verbose=0).ravel()
        return self.m.predict_proba(X)

    def summary_text(self):
        if BACKEND.startswith("Keras"):
            lines = []
            self.m.summary(print_fn=lines.append)
            return "\n".join(lines)
        return f"NumPy MLP: {self.hidden}, dropout={self.dropout}, lr={self.lr}"


def tune(Xtr, ytr, Xva, yva):
    """Basic grid search on the VALIDATION set (test set is never touched)."""
    grid = {"hidden": [(16, 8), (32, 16), (64, 32)], "dropout": [0.2, 0.3], "lr": [1e-3,
1e-2]}
    rows = []
    for hidden, dr, lr in itertools.product(grid["hidden"], grid["dropout"], grid["lr"]):
        mdl = Model(Xtr.shape[1], hidden, dr, lr)
        hist = mdl.fit(Xtr, ytr, Xva, yva)
        rows.append({"hidden_units": str(hidden), "dropout": dr, "learning_rate": lr,
                     "best_val_loss": round(min(hist["val_loss"]), 4),
                     "best_val_acc": round(max(hist["val_accuracy"] if "val_accuracy" in
hist else hist["val_acc"]), 4),
                     "epochs_run": len(hist["loss"])})
        print(rows[-1])
    res = pd.DataFrame(rows).sort_values("best_val_loss").reset_index(drop=True)
    res.to_csv(RESULTS_DIR / "hyperparameter_tuning.csv", index=False)
    best = res.iloc[0]
    return eval(best["hidden_units"]), float(best["dropout"]),
float(best["learning_rate"]), res
```

```
def permutation_importance(model, X, y, feats, n_rep=20):
    """Drop in accuracy when a feature is shuffled (model-agnostic feature
importance)."""
    rng = np.random.default_rng(SEED)
    base = accuracy_score(y, model.predict_proba(X) > .5)
    imp = []
    for j in range(X.shape[1]):
        drops = []
        for _ in range(n_rep):
            Xp = X.copy(); Xp[:, j] = rng.permutation(Xp[:, j])
            drops.append(base - accuracy_score(y, model.predict_proba(Xp) > .5))
        imp.append(np.mean(drops))
    return pd.Series(imp, index=feats).sort_values(ascending=False)



def plots(hist, y_te, p_te, imp, rf_imp):
    ep = np.arange(1, len(hist["loss"]) + 1)
    acc_k, vacc_k = ("accuracy", "val_accuracy") if "val_accuracy" in hist else
("acc", "val_acc")


    fig, ax = plt.subplots(figsize=(7, 4.5))                              # R1 loss
    ax.plot(ep, hist["loss"], label="Training loss"); ax.plot(ep, hist["val_loss"],
label="Validation loss")
    ax.axvline(int(np.argmin(hist["val_loss"])) + 1, color="grey", ls="--",
label="Best epoch")
    ax.set(title="Loss vs Epoch", xlabel="Epoch", ylabel="Binary cross-entropy");
ax.legend()
    fig.tight_layout(); fig.savefig(FIG_DIR / "r1_loss_vs_epoch.png", dpi=150);
plt.close(fig)


    fig, ax = plt.subplots(figsize=(7, 4.5))                              # R2 accuracy
    ax.plot(ep, hist[acc_k], label="Training accuracy"); ax.plot(ep, hist[vacc_k],
label="Validation accuracy")
    ax.set(title="Accuracy vs Epoch", xlabel="Epoch", ylabel="Accuracy",
ylim=(.85, 1.005)); ax.legend()
    fig.tight_layout(); fig.savefig(FIG_DIR / "r2_accuracy_vs_epoch.png",
dpi=150); plt.close(fig)


    cm = confusion_matrix(y_te, p_te > .5)                                # R3 confusion
matrix
    fig, ax = plt.subplots(figsize=(5.2, 4.5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES, cbar=False,
                annot_kws={"size": 16}, ax=ax)
    ax.set(title="Confusion matrix (test set)", xlabel="Predicted", ylabel="Actual")
    fig.tight_layout(); fig.savefig(FIG_DIR / "r3_confusion_matrix.png", dpi=150);
plt.close(fig)


    fpr, tpr, _ = roc_curve(y_te, p_te); roc_auc = auc(fpr, tpr)          # R4 ROC
    fig, ax = plt.subplots(figsize=(5.8, 5))
    ax.plot(fpr, tpr, lw=2, label=f"MLP (AUC = {roc_auc:.3f})"); ax.plot([0, 1], [0,
1], "k--", label="Chance")
    ax.set(title="ROC curve (test set)", xlabel="False positive rate", ylabel="True
positive rate"); ax.legend(loc="lower right")
    fig.tight_layout(); fig.savefig(FIG_DIR / "r4_roc_curve.png", dpi=150);
plt.close(fig)


    fig, ax = plt.subplots(1, 2, figsize=(13, 5.5))                       # R5 feature
importance
    t = imp.head(10)[::-1]; ax[0].barh(t.index, t.values, color="#e76f51");
ax[0].set_title("MLP permutation importance (top 10)")
    ax[0].set_xlabel("Mean drop in validation accuracy")
    t = rf_imp.head(10)[::-1]; ax[1].barh(t.index, t.values, color="#2a9d8f");
ax[1].set_title("Random Forest importance (top 10)")
    fig.tight_layout(); fig.savefig(FIG_DIR / "r5_feature_importance.png",
dpi=150); plt.close(fig)
    return float(roc_auc)



def main():
    Xtr, ytr, Xva, yva, Xte, yte, feats = load_data()
    print("Backend:", BACKEND)


    hidden, dropout, lr, tuning = tune(Xtr, ytr, Xva, yva)                # hyper-
parameter tuning
    model = Model(Xtr.shape[1], hidden, dropout, lr)                      # final model
    hist = model.fit(Xtr, ytr, Xva, yva)


    p_te = model.predict_proba(Xte); pred = (p_te > .5).astype(int)
    imp = permutation_importance(model, Xva, yva, feats)

    rf = RandomForestClassifier(300, random_state=SEED).fit(Xtr, ytr)
    rf_imp = pd.Series(rf.feature_importances_,
index=feats).sort_values(ascending=False)
    roc_auc = plots(hist, yte, p_te, imp, rf_imp)


    # save training history + test predictions (used by extra_analysis.py and the
report)
    (RESULTS_DIR / "history.json").write_text(json.dumps(hist))
    pd.DataFrame({"test_row": np.arange(len(yte)), "y_true": yte.astype(int),
"prob_benign": p_te, "y_pred": pred}
                 ).to_csv(RESULTS_DIR / "test_predictions.csv", index=False)


    lr_model = LogisticRegression(max_iter=1000).fit(Xtr, ytr)            # baseline
for comparison
    base_p = lr_model.predict_proba(Xte)[:, 1]
    tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
    metrics = {
        "backend": BACKEND,
        "architecture": {"input": len(feats), "hidden": list(hidden), "dropout": dropout,
"learning_rate": lr,
                         "output": "1 sigmoid unit", "optimizer": "Adam", "loss": "binary
cross-entropy",
                         "batch_size": BATCH, "max_epochs": EPOCHS,
"early_stopping_patience": PATIENCE},
        "epochs_run": len(hist["loss"]),
        "best_epoch": int(np.argmin(hist["val_loss"])) + 1,
        "test": {"accuracy": accuracy_score(yte, pred), "precision":
precision_score(yte, pred),
                 "recall": recall_score(yte, pred), "f1": f1_score(yte, pred), "auc":
roc_auc,
                 "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)},
        "baseline_logreg_test": {"accuracy": accuracy_score(yte, base_p > .5), "auc":
float(auc(*roc_curve(yte, base_p)[:2]))},
        "random_forest_test": {"accuracy": accuracy_score(yte, rf.predict(Xte)),
                               "auc": float(auc(*roc_curve(yte, rf.predict_proba(Xte)[:,
1])[:2]))},
        # accuracies at the best epoch (the weights that were restored by early
stopping)
        "final_train_acc": hist.get("accuracy",
hist.get("acc"))[int(np.argmin(hist["val_loss"]))],
        "final_val_acc": hist.get("val_accuracy",
hist.get("val_acc"))[int(np.argmin(hist["val_loss"]))],
        "top_features_mlp": imp.head(5).round(4).to_dict(),
        "top_features_rf": rf_imp.head(5).round(4).to_dict(),
        "tuning_top3": tuning.head(3).to_dict(orient="records"),
        "tuning_n_configs": int(len(tuning)),
        "classification_report": classification_report(yte, pred,
target_names=CLASS_NAMES, output_dict=True),
        "model_summary": model.summary_text(),
    }
    (RESULTS_DIR / "metrics.json").write_text(json.dumps(metrics, indent=2,
default=float))
    print(json.dumps({k: v for k, v in metrics.items() if k not in ("model_summary",
"classification_report")}, indent=2, default=float))



if __name__ == "__main__":
    main()
```